



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

UNIVERSIDAD POLITÉCNICA DE VALENCIA

Escuela Técnica Superior de Ingeniería Informática

Departamento de Sistemas Informáticos y Computación

Trabajo Fin de Máster

Arquitectura multiagente para la toma de decisiones en mercados de activos digitales

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas
e Imagen Digital

Autor

Shiyi Cheng

Tutora

Eva Onaindia de la Rivaherrera

Curso 2025–2026

Resumen

Los mercados de activos digitales mueven cientos de millones anuales y, aun así, carecen de herramientas analíticas y de automatizaciones integradas o de libre uso. La motivación principal de este proyecto nace de esta profunda brecha tecnológica. Actualmente, la gestión de carteras en la mayoría de los mercados se realiza de forma manual o mediante herramientas estáticas, lo que resulta altamente ineficiente y arriesgado.

El propósito principal de este TFM es evaluar si un ecosistema descentralizado de agentes de inteligencia artificial puede superar las limitaciones humanas y generar rendimientos estables en mercados de alta fricción. Se opta por una arquitectura multiagente de aprendizaje por refuerzo para descentralizar la toma de decisiones y reducir la complejidad computacional del mercado. El objetivo es que los agentes aprendan estrategias de inversión óptimas que maximicen el saldo total de la cartera.

La metodología desarrollada en el TFM se aplica al ámbito de la economía virtual de Counter-Strike 2, un popular videojuego que cuenta con un ecosistema financiero integrado con una alta frecuencia de intercambios.

Palabras clave: Mercados de activos digitales, toma de decisiones, arquitectura multiagente, aprendizaje por refuerzo.

Resum

Els mercats d'actius digitals mouen centenars de milions anuals i, tot i això, encara no disposen d'eines analítiques ni d'automatitzacions integrades o de lliure ús. La motivació principal d'aquest projecte naix d'aquesta profunda bretxa tecnològica. Actualment, la gestió de carteres en la majoria dels mercats es realitza de forma manual o mitjançant eines estàtiques, la qual cosa resulta altament ineficient i arriscada.

El propòsit principal d'aquest TFM és avaluar si un ecosistema descentralitzat d'agents d'intel·ligència artificial pot superar les limitacions humanes i generar rendiments estables en mercats d'alta fricció. S'opta per una arquitectura multiagent d'aprenentatge per reforç per descentralitzar la presa de decisions i reduir la complexitat computacional del mercat. L'objectiu és que els agents aprenguen estratègies d'inversió òptimes que maximitzen el saldo total de la cartera.

La metodologia desenvolupada en el TFM s'aplica a l'àmbit de l'economia virtual de Counter-Strike 2, un videojoc popular que compta amb un ecosistema financer integrat d'alta freqüència i liquiditat.

Paraules clau: Mercats d'actius digitals, presa de decisions, arquitectura multiagent, aprenentatge per reforç.

Abstract

Digital asset markets move hundreds of millions annually and yet still lack integrated or freely available analytical tools and automation systems. The main motivation of this project stems from this deep technological gap. At present, portfolio management in most markets is carried out manually or through static tools, which makes it highly inefficient and risky.

The main purpose of this master's thesis is to evaluate whether a decentralized ecosystem of artificial intelligence agents can overcome human limitations and generate stable returns in high-friction markets. A multi-agent reinforcement learning architecture is chosen to decentralize decision-making and reduce the computational complexity of the market. The goal is for the agents to learn optimal investment strategies that maximize the total portfolio balance.

The methodology developed in this master's thesis is applied to the virtual economy of Counter-Strike 2, a popular video game with an integrated financial ecosystem characterized by high frequency and liquidity.

Key words: Digital asset markets, decision-making, multi-agent architecture, reinforcement learning.

Índice general

Índice de figuras	7
Índice de tablas	8
Lista de algoritmos	9
Siglas y abreviaturas	10
Glosario	12
1 Introducción	15
1.1 Motivación	15
1.2 Planteamiento del problema	16
1.3 Objetivos	16
1.4 Estructura del documento	17
2 Estado del arte	18
2.1 Mercados y plataformas de activos digitales	18
2.2 Aprendizaje supervisado y no supervisado	20
2.3 Aprendizaje por refuerzo	21
2.4 Aprendizaje por refuerzo multiagente	23
2.5 Gestión de carteras y riesgo	25
2.6 Aplicaciones y trabajos relacionados	26
3 Contextualización del caso de estudio: activos digitales en Counter-Strike 2	27
3.1 Justificación, alcance y limitaciones	27
3.2 Artículos y atributos de Counter-Strike 2	28
3.3 Mercados secundarios y condiciones de intercambio	29
4 Arquitectura multiagente propuesta	31
4.1 Formulación técnica del sistema	31
4.2 Visión general del sistema	31
4.3 Flujo operativo	32
4.3.1 Clasificación de oportunidades	32
4.4 Definición del entorno	33
4.4.1 Formulación como MDP	33
4.4.2 Estado y observaciones	33
4.4.3 Espacio de acciones	34
4.4.4 Transición del entorno	35
4.5 Agentes del sistema	35
4.5.1 Scout	35
4.5.2 Trader	35
4.5.3 Portfolio	36
4.6 Función de recompensa	36
4.6.1 Recompensa individual	36
4.6.2 Recompensa cooperativa	36

4.7	Restricciones de riesgo	37
5	Configuración del entorno y datos	38
5.1	Fuentes de datos	38
5.1.1	Mercado de Steam	38
5.1.2	BUFF y mercados externos	38
5.1.3	OCR e importación manual	39
5.2	Adquisición y persistencia	40
5.2.1	Pipeline de adquisición	40
5.2.2	Modelo de persistencia	41
5.3	Preparación de características	41
5.3.1	Variables de mercado	41
5.3.2	Dataset temporal de trading	42
5.3.3	Variables de cartera	42
5.4	Reglas económicas migradas	43
5.5	Simulador de cartera	43
5.5.1	Operaciones permitidas	43
5.5.2	Comisiones y valoración	43
6	Implementación y operación	45
6.1	Organización del sistema	45
6.2	Flujo operativo de adquisición	45
6.2.1	Sesiones, scraping y trazabilidad	45
6.2.2	Persistencia incremental	46
6.3	Consola de consulta	46
6.4	Reproducibilidad y configuración	47
7	Experimentación y análisis	48
7.1	Diseño experimental	48
7.1.1	Escenarios de evaluación	48
7.1.2	Estrategias de comparación	49
7.2	Validación técnica y pruebas	49
7.3	Métricas	50
7.3.1	Métricas de rentabilidad	50
7.3.2	Métricas de riesgo	50
7.4	Resultados	50
7.4.1	Migración de reglas económicas	50
7.4.2	Resultados del modelo supervisado	51
7.4.3	Evolución de los datasets de trading	51
7.4.4	Protocolo temporal reproducible	52
7.4.5	Ablación de histórico y aumentación	54
7.4.6	Comprobación del corte reciente	55
7.4.7	Evidencia de pruebas automatizadas	56
7.4.8	Validación funcional de OCR	56
7.4.9	Dataset de trading operativo	57
7.4.10	Dificultades y pivotaciones	58
7.4.11	Protocolo de episodios MARL	58
7.4.12	Resultados MARL	59
7.5	Experimentos finales pendientes	60

7.6	Discusión	60
7.6.1	Amenazas a la validez	60
7.6.2	Casos favorables	60
7.6.3	Limitaciones observadas	61
7.6.4	Lectura de los resultados	61
8	Conclusiones	62
8.1	Conclusiones principales	62
8.2	Cumplimiento de objetivos	63
8.3	Limitaciones	63
8.4	Trabajo futuro	63
	Bibliografía	65
A	Anexos	68
A.1	Estructura del repositorio	68
A.2	Configuración del entorno	69
A.3	Comandos de datasets y modelos	69
A.4	Parámetros de entrenamiento	69
A.5	Modelo de datos	69
A.6	Resultados adicionales	70

Índice de figuras

2.1	Ciclo de interacción entre un agente y un entorno en aprendizaje por refuerzo.	22
2.2	Interacción general en MARL con entrenamiento centralizado y ejecución descentralizada.	24
4.1	Capas del sistema.	32
7.1	Evolución de cortes.	52
7.2	Corte temporal de marzo.	53
7.3	Modelos en marzo.	54
7.4	Histórico y aumentación.	55
7.5	Métricas por umbral.	56
A.1	Estructura del repositorio.	68

Índice de tablas

2.1	Ejemplos de mercados integrados.	19
2.2	Ejemplos de plataformas externas.	20
4.1	Roles del sistema.	35
5.1	Controles del pipeline OCR.	40
5.2	Modelo de datos.	41
5.3	Reglas de simulación.	43
6.1	Vistas de consola.	47
7.1	Matriz experimental.	48
7.2	Cobertura de pruebas.	50
7.3	Umbrales de dirección.	51
7.4	Cortes de datos.	52
7.5	Configuraciones de marzo.	53
7.6	Comparativa de marzo.	54
7.7	Ablación logística.	55
7.8	Corte BUFF–Steam.	56
7.9	Pruebas automatizadas.	57
7.10	Validación funcional de OCR.	57
7.11	Episodio MARL.	58
7.12	Ejecuciones MARL.	59
A.1	Configuración experimental.	70

Lista de algoritmos

1	Ciclo de decisión	32
2	Decisión cooperativa	34
3	Extracción OCR.	40
4	Dataset temporal.	42
5	Flujo de scraping.	46
6	Ejecución de experimento.	47
7	Entrenamiento PPO	49

Siglas y abreviaturas

Esta relación recoge únicamente las siglas, códigos y abreviaturas que se emplean de forma recurrente.

API Interfaz que permite que dos programas intercambien datos o funciones.

BUFF Plataforma externa de compraventa de activos digitales, usada como fuente de precios en el caso de estudio.

CLAHE Técnica de mejora de contraste de una imagen por zonas, del inglés *Contrast Limited Adaptive Histogram Equalization*.

CLI Interfaz de línea de comandos, del inglés *Command-Line Interface*.

CNY Código internacional del yuan chino.

CS2 *Counter-Strike 2*, videojuego empleado como caso de estudio.

CSV Formato de datos tabulares separado por comas, del inglés *Comma-Separated Values*.

CTCE Entrenamiento centralizado y ejecución centralizada, del inglés *Centralized Training with Centralized Execution*.

CTDE Entrenamiento centralizado y ejecución descentralizada, del inglés *Centralized Training with Decentralized Execution*.

EUR Código internacional del euro.

F1 Métrica que combina precisión y exhaustividad mediante su media armónica.

IA Inteligencia artificial.

MADDPG Algoritmo de aprendizaje por refuerzo multiagente, del inglés *Multi-Agent Deep Deterministic Policy Gradient*.

MAPPO Versión multiagente de PPO, del inglés *Multi-Agent Proximal Policy Optimization*.

MARL Aprendizaje por refuerzo multiagente, del inglés *Multi-Agent Reinforcement Learning*.

MDP Modelo matemático de decisión secuencial, del inglés *Markov Decision Process*.

OCR Reconocimiento de texto en imágenes, del inglés *Optical Character Recognition*.

PPO Algoritmo de optimización de políticas, del inglés *Proximal Policy Optimization*.

PR-AUC Área bajo la curva precisión-exhaustividad, del inglés *Precision-Recall Area Under the Curve*.

RL Aprendizaje por refuerzo, del inglés *Reinforcement Learning*.

RLlib Biblioteca de Ray utilizada para entrenar y evaluar políticas de aprendizaje por refuerzo.

ROC-AUC Área bajo la curva ROC, del inglés *Receiver Operating Characteristic Area Under the Curve*.

RSI Indicador de la intensidad de los cambios de precio, del inglés *Relative Strength Index*.

TFM Trabajo Fin de Máster.

Glosario

Este glosario reúne los términos de mercado y decisión empleados de forma recurrente en la memoria.

Activo digital Elemento asociado a una cuenta de usuario o a un servicio digital que puede utilizarse, coleccionarse o intercambiarse.

Agente Programa que toma decisiones a partir de la información que recibe.

Aprendizaje por refuerzo Método de aprendizaje en el que un agente observa un entorno, selecciona acciones y recibe recompensas que permiten evaluar sus consecuencias.

Aprendizaje por refuerzo multiagente Extensión del aprendizaje por refuerzo en la que dos o más agentes interactúan con un mismo entorno y pueden cooperar, competir o decidir de forma independiente.

Beneficio neto Resultado económico de una operación después de descontar el precio de compra, las comisiones y los costes aplicables.

Capital bloqueado Parte del saldo que permanece comprometida en una compra o que no puede reutilizarse mientras el activo no pueda venderse o transferirse.

Cartera Conjunto formado por el saldo disponible y las posiciones abiertas, es decir, los activos digitales adquiridos que aún no se han vendido.

Demanda Número de personas usuarias que quieren adquirir un activo digital.

Disponibilidad Número de unidades de un activo digital que están a la venta en una plataforma.

Economía de videojuego Conjunto de reglas con las que un videojuego permite obtener, poseer, utilizar e intercambiar activos digitales.

Economía persistente Economía de videojuego en la que los activos digitales permanecen asociados a la cuenta de la persona usuaria después de terminar una partida.

Entorno Representación de la situación en la que un agente toma decisiones, formada por la información disponible y las reglas que determinan las consecuencias de sus acciones.

Escasez Número de unidades existentes de un activo digital en todo el mundo.

Exposición Parte de la cartera comprometida con un activo, una plataforma o una categoría de activos.

Horizonte temporal Periodo previsto entre la compra de un activo y su posible venta.

Mercado cerrado Entorno en el que los activos solo se utilizan dentro del videojuego y no se intercambian entre personas usuarias.

Mercado centralizado Mercado en el que una empresa o plataforma administra las cuentas, los activos digitales o las reglas de intercambio.

Mercado descentralizado Entorno en el que la gestión del activo y las reglas de intercambio no dependen de una plataforma central.

Mercado digital Entorno en línea en el que las personas usuarias intercambian bienes o servicios mediante plataformas.

Mercado integrado Mercado administrado por el propio ecosistema del videojuego o de distribución, que controla los activos digitales asociados a las cuentas y las reglas de intercambio.

Mercado secundario Mercado de reventa entre personas usuarias.

Oferta Publicación mediante la cual una persona pone a la venta un activo digital identificado por sus características relevantes e indica su precio.

Operación Decisión sobre un activo digital. Puede consistir en comprar, vender, mantener una posición abierta o descartar una compra o una venta.

Operación de compraventa Análisis conjunto de una compra de un activo digital y su venta posterior.

Orden de compra Publicación que indica el precio máximo que una persona está dispuesta a pagar por un activo digital concreto.

Plataforma Servicio digital donde se administran las reglas del intercambio y la información necesaria para participar en él.

Plataforma externa Plataforma distinta del servicio que administra originalmente los activos digitales vinculados a la cuenta y que publica ofertas o facilita operaciones sobre activos transferibles.

Posición abierta Activo digital ya adquirido que permanece en la cartera hasta su venta. Mientras la posición está abierta, el importe empleado en la compra no puede destinarse a otra operación.

Rareza Dificultad para obtener un activo digital.

Recompensa Señal numérica que evalúa la consecuencia de una acción en aprendizaje por refuerzo y guía el aprendizaje de una política.

Rentabilidad Relación entre el beneficio neto de una operación y el capital empleado para realizarla.

Spread Diferencia entre el mejor precio de venta y la mejor orden de compra disponibles para un mismo activo en un momento dado.

Trade hold Bloqueo temporal que impide vender o transferir un activo digital recién adquirido.

Volumen de intercambios Cantidad de unidades de un activo digital vendidas durante un periodo, como un día, una semana o un mes.

Introducción

Los **mercados digitales** son entornos en línea en los que las personas usuarias intercambian bienes o servicios mediante **plataformas**. Una plataforma administra las reglas del intercambio y la información necesaria para participar en él. Cuando los bienes están asociados a una cuenta o a un servicio digital y pueden utilizarse, coleccionarse o intercambiarse, se denominan **activos digitales**.

La gestión de activos digitales plantea decisiones sucesivas de compra, venta, mantenimiento o descarte. Estas decisiones se denominan **operaciones**. Cada operación puede modificar la **cartera**, formada por el saldo disponible y las **posiciones abiertas**, es decir, los activos digitales adquiridos que aún no se han vendido. Por ello, el objetivo no es solo identificar una diferencia de precios, sino evaluar decisiones que afectan a la evolución de la cartera con el fin de generar un beneficio económico.

El **aprendizaje por refuerzo** (RL), del inglés *Reinforcement Learning*, es un paradigma para abordar problemas de decisión secuencial. En este marco, un **agente** interactúa con un **entorno**, selecciona acciones y recibe **recompensas**. A diferencia de un modelo que solo estima un resultado futuro, RL permite estudiar cómo una decisión actual condiciona las decisiones posteriores.

El **aprendizaje por refuerzo multiagente** (MARL), del inglés *Multi-Agent Reinforcement Learning*, amplía este marco cuando varios agentes interactúan con el mismo entorno. Los agentes pueden cooperar, competir o actuar de forma independiente. La coordinación puede ser centralizada, cuando una única entidad toma la decisión conjunta, o descentralizada, cuando cada agente decide desde su propia información y responsabilidad.

1.1 Motivación

Los mercados de activos digitales en videojuegos permiten que las personas usuarias compren, vendan e intercambien objetos asociados a un videojuego. El valor de estos activos digitales puede depender de su utilidad (por ejemplo, aumenta la vida de un personaje), apariencia, **rareza** (la dificultad para obtener un activo digital), **disponibilidad** (el número de unidades en venta) y **demandas** (el número de personas que quieren adquirirlo). La literatura sobre bienes virtuales identifica dimensiones funcionales, estéticas y sociales como determinantes de ese valor [1]. Por ello, una misma operación puede presentar un resultado económico diferente según el momento y la plataforma en los que se realice.

La motivación principal de este trabajo es apoyar la identificación de operaciones que puedan generar un **beneficio neto** (el resultado después de descontar el precio de compra, las comisiones y otros costes). Una decisión acertada puede aportar un resultado económico favorable a la persona usuaria. Sin embargo, ese resultado no está

garantizado y comparar dos precios no es suficiente para decidir. Un mismo activo digital puede aparecer en varias plataformas con precios, divisas, comisiones y condiciones de intercambio distintas. La decisión exige valorar cuánto se pagaría al comprar, cuánto se recibiría al vender y si se aplica un *trade hold* (bloqueo temporal que impide vender o transferir el activo adquirido).

1.2 Planteamiento del problema

Las decisiones de compra y venta se suceden en el tiempo. Cada compra utiliza una parte del saldo disponible de la cartera digital. Mientras el activo digital adquirido permanezca en la cartera y no se pueda vender, ese importe no puede utilizarse para adquirir otro activo digital. Por ello, el problema no consiste solo en comparar precios, sino también en decidir cómo emplear un saldo limitado entre distintas posibilidades de compra.

Las plataformas no siempre emplean la misma moneda, el mismo nombre ni la misma fecha de observación para un activo digital. Antes de comparar precios, es necesario unificar esa información.

Cuando se analizan conjuntamente una compra y una venta posterior, se habla de una **operación de compraventa**. El punto de partida es un procedimiento manual en el que, para cada activo digital, se anotan el precio de compra, el precio de venta, la conversión de moneda, las comisiones, la fecha de desbloqueo y el **capital bloqueado** (el saldo que no puede utilizarse para nuevas compras). Esta primera implementación manual de donde partimos permite evaluar casos concretos, pero obliga a repetir los cálculos y a comprobar los datos de cada activo digital.

Con la información mencionada anteriormente, el sistema debe decidir qué operaciones conviene realizar, mantener o descartar. Para justificar esa decisión, también debe considerar el **volumen de intercambios** (la cantidad de unidades vendidas durante un día, una semana o un mes), las reglas económicas y de intercambio aplicables y el estado de la cartera. La pregunta que debe responder es si la posible operación merece ser estudiada con criterios consistentes.

1.3 Objetivos

El objetivo principal de este TFM es diseñar y evaluar, mediante simulación, una arquitectura multiagente que apoye la identificación y evaluación de operaciones orientadas a generar un beneficio neto en mercados de activos digitales. El caso de estudio es la economía virtual del videojuego *Counter-Strike 2*.

Los objetivos específicos se agrupan en cuatro bloques:

- **Análisis del caso de estudio:** analizar el mercado de activos digitales de *Counter-Strike 2*, sus plataformas, restricciones, comisiones, divisas, volumen de intercambios y efectos del *trade hold*.
- **Construcción de datos:** construir un flujo de adquisición y preparación de datos procedentes de diferentes plataformas de mercado y fuentes manuales, manteniendo la trazabilidad de la fuente, la fecha, la moneda y la calidad del dato.

- **Simulación y estimación:** desarrollar un simulador que incorpore las reglas económicas del procedimiento manual, construir conjuntos de datos temporales y obtener estimaciones de beneficio que puedan incorporarse al entorno de decisión.
- **Coordinación multiagente:** formular un entorno de aprendizaje por refuerzo con el estado de la cartera, las acciones y las recompensas definidos. Sobre esa base, distribuir los roles de detección, decisión y control de riesgo entre agentes especializados y evaluar su funcionamiento con métricas de **rentabilidad**, riesgo, **exposición** y capital bloqueado.

1.4 Estructura del documento

La memoria se estructura de forma secuencial siguiendo las fases principales del proyecto. A continuación, se detalla el contenido de cada capítulo:

- **Capítulo 1:** introduce el contexto del proyecto, la motivación, el problema abordado, la solución propuesta, los objetivos, el alcance y la estructura.
- **Capítulo 2:** presenta el estado del arte sobre mercados de activos digitales, plataformas de intercambio, aprendizaje por refuerzo, coordinación multiagente y toma de decisiones en carteras.
- **Capítulo 3:** analiza los conceptos operativos del mercado de activos digitales en videojuegos y justifica el uso de *Counter-Strike 2* como caso de estudio.
- **Capítulo 4:** describe la arquitectura multiagente propuesta. Se presenta la formulación técnica del sistema, flujo general, entorno de decisión, agentes, función de recompensa y restricciones de riesgo.
- **Capítulo 5:** expone el entorno de datos y simulación. Se detallan fuentes de adquisición, persistencia, características utilizadas, simulador de cartera, OCR e interfaz de consulta.
- **Capítulo 6:** describe la implementación y operación del sistema. Presenta la organización del repositorio, el scraping, la persistencia, la consola local y la configuración reproducible.
- **Capítulo 7:** recoge la experimentación realizada. Incluye migración de reglas económicas, construcción de datasets supervisados, evaluación de modelos tabulares, pruebas técnicas y de rendimiento, dataset de trading, estado del entrenamiento MARL, dificultades y pivotaciones detectadas.
- **Capítulo 8:** presenta conclusiones principales, revisa el cumplimiento de objetivos y plantea líneas de trabajo necesarias para completar la validación experimental.
- **Anexos:** reúnen información complementaria sobre estructura del repositorio, configuración del entorno, comandos de ejecución, parámetros de entrenamiento y modelo de datos.

Estado del arte

Este capítulo presenta los fundamentos teóricos y los trabajos previos que contextualizan este trabajo. En primer lugar, sitúa los mercados digitales de activos dentro de las economías de los videojuegos y diferencia los principales tipos de mercado y plataforma. Esta distinción permite entender que las posibilidades de intercambio, los precios observados y las reglas de cada operación dependen del entorno en el que se realiza.

Después, el capítulo introduce el aprendizaje supervisado como método para obtener estimaciones a partir de datos históricos. A continuación, presenta los fundamentos del aprendizaje por refuerzo en el caso de un único agente, incluyendo la interacción entre agente y entorno, los procesos de decisión de Markov, las políticas, las recompensas y el retorno acumulado. Estos conceptos se amplían después al aprendizaje por refuerzo multiagente, donde varios agentes actúan sobre un entorno compartido. Se revisan las formas de cooperación, las recompensas individuales y colectivas, y los enfoques centralizados y descentralizados.

Por último, se exponen los principios generales de la toma de decisiones en carteras y se revisan aplicaciones relacionadas de aprendizaje por refuerzo y aprendizaje por refuerzo multiagente. Esta revisión permite identificar las aportaciones, limitaciones y retos que fundamentan el planteamiento desarrollado en los capítulos posteriores.

2.1 Mercados y plataformas de activos digitales

Los bienes virtuales pueden tener utilidad dentro de un videojuego y, cuando las reglas lo permiten, circular entre personas usuarias a través de mecanismos de intercambio. La investigación sobre consumo de bienes virtuales muestra que la demanda no depende solo de una función dentro del juego. También intervienen la apariencia, la rareza, la identidad social y la percepción de valor del objeto [1, 2].

Por ejemplo, un atuendo para un avatar o una apariencia para un arma modifica su presentación visual sin cambiar las reglas de juego. Un objeto funcional puede dar acceso a una capacidad, una zona o un recurso dentro del juego, proporcionando ventaja al usuario. También existen objetos coleccionables cuyo interés depende de una serie limitada, su rareza o la demanda de la comunidad. Algunos bienes son consumibles, como cajas, sobres o mejoras, y desaparecen o se transforman al utilizarlos.

En los mercados de activos digitales hay dos formas de anunciar un intercambio:

- Una persona usuaria que posee un activo digital y desea venderlo publica una **oferta**. En esa publicación ofrece el activo digital e indica el precio de venta para que otra persona pueda comprarlo.

- Una persona usuaria que desea adquirir un activo digital publica una **orden de compra**. En esa publicación ofrece una cantidad máxima de dinero por un activo digital concreto.

Cuando se completa el intercambio, la plataforma o el servicio de pago puede cobrar costes de transacción, como una comisión por la compra o la venta. Las reglas de cada plataforma determinan cómo se publican, consultan y aceptan estas publicaciones.

Los mercados de activos digitales pueden clasificarse en centralizados y descentralizados según quién administra la cuenta, los activos digitales asociados a ella y las reglas de intercambio:

- **Mercados centralizados:** una empresa o plataforma administra las cuentas, los activos digitales o las reglas de intercambio. Dentro de esta categoría se encuentran:
 - **Mercados cerrados:** los activos digitales se usan dentro del videojuego, pero no se intercambian entre personas usuarias. Por tanto, no constituyen un **mercado secundario** (un mercado en el que las personas usuarias se revenden activos digitales entre sí) ni existe un precio de reventa observable.
 - **Mercados integrados:** el mercado está incluido dentro de la plataforma que administra la cuenta y los activos digitales asociados a ella. Esa plataforma establece las reglas para publicar ofertas y órdenes de compra.
 - **Plataformas externas:** proporcionan un espacio de intercambio distinto de la plataforma donde se gestionan originalmente la cuenta y los activos digitales. Aplican sus propias reglas, precios y mecanismos de pago.
- **Mercados descentralizados:** la gestión del activo y las reglas de intercambio no dependen de una plataforma central. Habitualmente emplean una infraestructura distribuida para registrar la propiedad y las operaciones.

Los mercados integrados están presentes en distintos ecosistemas de videojuego y distribución. La Tabla 2.1 presenta algunos ejemplos.

Mercado	Ecosistema	Activos habituales
Steam Community Market	Steam	Activos cosméticos, cajas y pegatinas
Roblox Marketplace	Roblox	Prendas digitales, accesorios y animaciones
Auction House	World of Warcraft	Equipo, materiales y moneda
EVE Online Market	EVE Online	Naves, módulos y recursos

Tabla 2.1: Ejemplos de mercados integrados.

Aunque sus activos digitales y reglas son diferentes, estos mercados gestionan los activos digitales asociados a las cuentas y el mecanismo de intercambio dentro del mismo ecosistema [3, 4, 5, 6].

Las plataformas externas ofrecen espacios de intercambio independientes para activos digitales transferibles. La Tabla 2.2 presenta algunos ejemplos.

Plataforma	Especialización	Modalidad
BUFF	Activos digitales transferibles	Compra y venta entre usuarios
Skinport	Activos cosméticos de videojuegos	Mercado con precios publicados
BitSkins	Activos digitales transferibles	Mercado de terceros
DMarket	Activos digitales de videojuegos	Compra y venta entre usuarios

Tabla 2.2: Ejemplos de plataformas externas.

Las dos tablas no deben interpretarse como pares equivalentes, ya que cada una reúne ejemplos independientes de una configuración de intercambio [7, 8, 9, 10].

2.2 Aprendizaje supervisado y no supervisado

El aprendizaje supervisado es un método que aprende a partir de ejemplos en los que se conoce tanto la información de entrada como el resultado que debe estimarse. Cada ejemplo se representa mediante una entrada x y una etiqueta y , que es el resultado conocido asociado a esa entrada. Durante el entrenamiento, el modelo ajusta sus parámetros para aprender la relación entre ambas variables.

Una vez entrenado, el modelo aplica la relación aprendida a nuevas entradas y genera una estimación:

$$\hat{y} = f_{\theta}(x), \quad (2.1)$$

donde x representa la información de entrada, f_{θ} el modelo con parámetros θ y $\hat{y}$ el resultado estimado. Cuando el resultado pertenece a una categoría, el problema se denomina clasificación. Cuando el resultado es un valor numérico, se denomina regresión. En un problema de clasificación binaria, el modelo también puede estimar la probabilidad de que una entrada pertenezca a la clase positiva:

$$\hat{p} = \mathbb{P}(Y = 1 \mid x). \quad (2.2)$$

En la Ecuación 2.2, Y representa el resultado que se desea clasificar, $Y = 1$ representa la clase positiva y $\hat{p}$ es la probabilidad estimada de pertenecer a esa clase. El entrenamiento consiste en seleccionar los parámetros que reducen el error entre las estimaciones y los resultados conocidos del conjunto de ejemplos. Si se dispone de n ejemplos, este ajuste puede expresarse mediante una función de pérdida $\mathcal{L}$:

$$\theta^* = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f_{\theta}(x_i)). \quad (2.3)$$

En la Ecuación 2.3, n es el número de ejemplos, x_i es la entrada del ejemplo i , y_i su resultado conocido y $\mathcal{L}$ mide el error entre ese resultado y la estimación del modelo. El valor θ^* representa los parámetros que minimizan el error medio. La ecuación no fija un único modelo ni una única forma de medir el error. La función de pérdida se elige según la tarea. Por ejemplo, puede penalizar la diferencia entre un valor estimado y el valor observado en regresión, o penalizar una clasificación incorrecta cuando las etiquetas representan categorías.

El aprendizaje no supervisado parte de datos de entrada sin una etiqueta o resultado conocido asociado. Su objetivo es descubrir estructuras presentes en los datos, como grupos de observaciones con características parecidas, relaciones entre variables o comportamientos poco habituales. Por ello, puede utilizarse para explorar un conjunto de datos, agrupar elementos similares o detectar observaciones anómalas.

La diferencia esencial entre ambos enfoques se encuentra en la información disponible durante el entrenamiento. El aprendizaje supervisado aprende a relacionar una entrada con un resultado conocido, mientras que el aprendizaje no supervisado describe la estructura de los datos sin partir de un resultado objetivo. Ambos enfoques pueden utilizarse de forma complementaria, por ejemplo, cuando los grupos o patrones identificados mediante aprendizaje no supervisado se incorporan como información adicional a un modelo supervisado. Una estimación o una probabilidad no constituye por sí sola una decisión, ya que no incorpora automáticamente las restricciones ni las consecuencias posteriores de una acción.

2.3 Aprendizaje por refuerzo

El aprendizaje por refuerzo (RL, por sus siglas en inglés de *Reinforcement Learning*) estudia problemas de decisión secuencial en los que un agente aprende mediante su interacción con un entorno [11]. En cada instante t , el agente recibe la información disponible, denominada estado s_t , selecciona una acción a_t y recibe una recompensa r_{t+1} . Como consecuencia de esa acción, el entorno pasa a un nuevo estado s_{t+1} . A diferencia del aprendizaje supervisado, donde un modelo aprende a reproducir etiquetas históricas, en RL el agente aprende qué acciones conviene seleccionar a lo largo de una secuencia de decisiones.

Esta interacción puede representarse mediante un proceso de decisión de Markov, o MDP, definido por el estado, las acciones, la transición, la recompensa y el factor de descuento:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \quad 0 \leq \gamma < 1. \quad (2.4)$$

En la Ecuación 2.4, $\mathcal{M}$ representa el proceso de decisión de Markov, $\mathcal{S}$ es el conjunto de estados posibles y $\mathcal{A}$ el conjunto de acciones disponibles. La función de transición $P(s' | s, a)$ expresa la probabilidad de pasar del estado s al estado s' después de ejecutar la acción a . La función $R(s, a, s')$ asigna la recompensa obtenida en esa transición. El factor γ determina cuánto peso reciben las recompensas futuras frente a las inmediatas.

El comportamiento que aprende el agente se denomina política y se representa por π . En una política estocástica, $\pi(a | s)$ expresa la probabilidad de seleccionar la acción a cuando el agente se encuentra en el estado s :

$$\pi(a | s) = \mathbb{P}(A_t = a | S_t = s). \quad (2.5)$$

La Figura 2.1 representa el ciclo básico de interacción. El estado del entorno se entrega al agente, el agente selecciona una acción y el entorno actualiza el estado y proporciona la recompensa correspondiente.

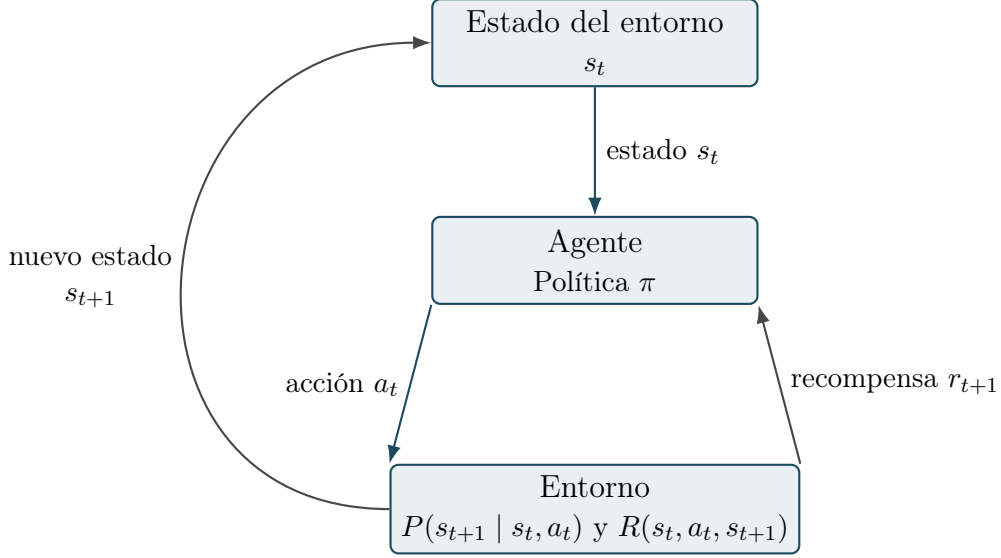


Figura 2.1: Ciclo de interacción entre un agente y un entorno en aprendizaje por refuerzo.

El retorno mide la recompensa acumulada durante un episodio. Si T representa el último instante del episodio, el retorno desde el instante t se define como:

$$G_t^\pi = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}. \quad (2.6)$$

En la Ecuación 2.6, r_{t+k+1} es la recompensa recibida después de la acción tomada en cada instante futuro y γ^k es el descuento aplicado a esa recompensa. Por tanto, una recompensa inmediata tiene más peso que una recompensa situada varios instantes después. El objetivo de RL es aprender la política que maximiza el retorno esperado desde el estado inicial:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} [G_0^\pi]. \quad (2.7)$$

En la Ecuación 2.7, π^* es la política buscada y $\mathbb{E}_{\pi}$ representa el valor medio del retorno al seguir la política π . La ecuación no busca únicamente la acción con una recompensa inmediata mayor. Busca una política que tenga en cuenta las consecuencias de las decisiones posteriores. La propiedad de Markov no implica que el agente conozca el futuro, sino que el estado reúne la información necesaria para decidir en el instante actual.

Durante el aprendizaje, el agente debe equilibrar exploración y explotación. La exploración consiste en probar acciones cuya utilidad todavía es incierta para conocer sus consecuencias. La explotación consiste en seleccionar las acciones que, según lo aprendido hasta ese momento, tienen un retorno esperado mayor. Explorar en exceso puede acumular decisiones poco útiles, mientras que explotar demasiado pronto puede impedir descubrir alternativas mejores.

2.4 Aprendizaje por refuerzo multiagente

Un sistema multiagente reúne dos o más agentes que actúan sobre un mismo entorno. Estos agentes pueden colaborar, competir o tomar decisiones independientes. En un sistema cooperativo comparten un objetivo global, aunque cada agente disponga de información y responsabilidades distintas [12, 13]. El aprendizaje por refuerzo multiagente, o MARL, amplía la formulación de RL para representar estos sistemas.

Un modelo genérico de MARL puede expresarse como una extensión del proceso de decisión de Markov:

$$\mathcal{M} = (\mathcal{N}, \mathcal{S}, \{\mathcal{A}^i\}_{i \in \mathcal{N}}, P, \{R^i\}_{i \in \mathcal{N}}, \gamma). \quad (2.8)$$

En la Ecuación 2.8, $\mathcal{N}$ es el conjunto de agentes, $\mathcal{S}$ es el conjunto de estados globales y $\mathcal{A}^i$ es el conjunto de acciones disponibles para el agente i . La función P describe la transición del entorno después de la acción conjunta de los agentes. Cada función R^i calcula la recompensa que recibe el agente i y γ es el factor de descuento. En cada instante, los agentes generan una acción conjunta:

$$\mathbf{a}_t = (a_t^1, \dots, a_t^N), \quad r_{t+1}^i = R^i(s_t, \mathbf{a}_t, s_{t+1}). \quad (2.9)$$

En la Ecuación 2.9, N es el número de agentes, $\mathbf{a}_t$ reúne las acciones tomadas en el instante t y r_{t+1}^i es la recompensa que recibe el agente i tras la transición al estado s_{t+1} . Esta formulación permite distinguir cómo se reparten los incentivos entre los agentes.

En un problema plenamente cooperativo todos los agentes reciben la misma recompensa:

$$r_t^1 = r_t^2 = \dots = r_t^N = r_t. \quad (2.10)$$

En la Ecuación 2.10, r_t representa la recompensa común. Esta estructura alinea explícitamente a todos los agentes con el mismo resultado. En un problema cooperativo, en cambio, cada agente puede recibir una recompensa individual distinta. Estas recompensas no tienen por qué coincidir, pero deben diseñarse para que los comportamientos que incentivan contribuyan al objetivo compartido. La distinción es relevante porque una recompensa común favorece la cooperación directa, mientras que las recompensas individuales permiten reflejar responsabilidades diferentes y pueden dificultar la coordinación si no están bien alineadas.

MARL introduce dificultades que no aparecen con un único agente. Cada agente aprende y modifica su política mientras los demás también modifican las suyas. Por ello, desde el punto de vista de un agente, las consecuencias de una acción pueden cambiar aunque el entorno físico permanezca igual. Este fenómeno se denomina no estacionariedad. Además, un agente puede decidir a partir de una observación local que no contiene toda la información del estado global, lo que dificulta atribuir a cada decisión la parte del resultado que le corresponde.

La información disponible puede repartirse de forma distinta durante el entrenamiento y la ejecución. El entrenamiento es la fase en la que las políticas se ajustan a partir de la experiencia obtenida en el entorno. La ejecución es la fase posterior en la que las

políticas ya entrenadas seleccionan acciones. En el entrenamiento centralizado y ejecución descentralizada, conocido como CTCE, una entidad central recibe el estado global y selecciona la acción conjunta tanto durante el entrenamiento como durante la ejecución. Esta configuración facilita la coordinación porque concentra toda la información en una única decisión, pero exige que el estado global y la entidad central estén disponibles cuando el sistema actúa. Por ejemplo, un coche autónomo puede utilizar un controlador central que recibe toda la información disponible y decide la aceleración, el frenado y la dirección tanto durante el entrenamiento como durante la conducción.

El paradigma de entrenamiento centralizado y ejecución descentralizada, conocido como CTDE, separa ambas funciones. Durante el entrenamiento, un modelo evaluador puede utilizar el estado global y las acciones de todos los agentes para estimar la calidad esperada de su coordinación. Durante la ejecución, cada agente selecciona su acción a partir de su propia observación local. Por ejemplo, varios coches autónomos pueden entrenarse con un modelo evaluador que conoce la posición, la velocidad y las acciones de todos ellos. Cuando circulan, cada coche decide utilizando únicamente sus propios sensores. Este enfoque permite entrenar con información adicional sin exigir que esa información esté disponible cuando el sistema toma decisiones [14, 15]. El número de coches no determina el paradigma. Varios coches controlados por una única entidad central seguirían un enfoque CTCE. Por tanto, CTCE y CTDE representan dos formas distintas de distribuir la información y la responsabilidad de decisión entre los agentes.

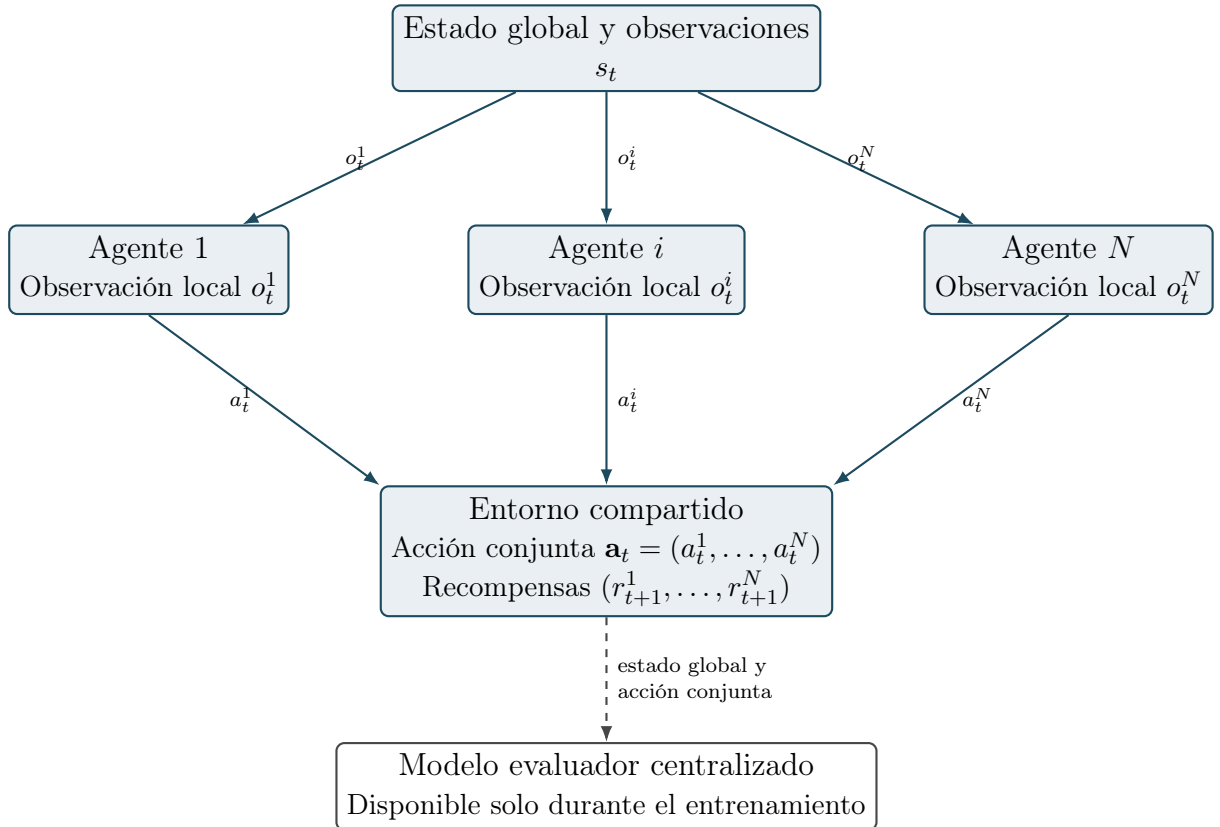


Figura 2.2: Interacción general en MARL con entrenamiento centralizado y ejecución descentralizada.

La Figura 2.2 muestra una interacción general de este tipo. Las líneas continuas representan el intercambio necesario para ejecutar la política de cada agente. Las líneas discontinuas representan la información adicional que utiliza el modelo evaluador únicamente durante el entrenamiento.

2.5 Gestión de carteras y riesgo

La gestión de una cartera estudia cómo las decisiones sobre varios activos y el saldo disponible afectan al conjunto de recursos. No basta con evaluar cada activo de forma aislada. Una decisión de compra o venta de un activo puede parecer atractiva por sí sola y, sin embargo, puede aumentar demasiado la dependencia de la cartera respecto a un activo, una categoría o una plataforma.

La teoría moderna de carteras plantea que la rentabilidad y el riesgo deben analizarse conjuntamente [16]. La rentabilidad expresa el resultado obtenido en relación con los recursos utilizados. El riesgo recoge la incertidumbre y las posibles variaciones desfavorables que pueden afectar a ese resultado. Por ello, la gestión de carteras busca equilibrar ambos elementos en lugar de maximizar únicamente el resultado esperado de una decisión individual.

La asignación de recursos determina qué parte de la cartera se dedica a cada alternativa y qué parte permanece disponible. Una cartera concentrada destina una proporción elevada de sus recursos a pocos activos, categorías o plataformas. Esta concentración puede aumentar el resultado si los activos seleccionados evolucionan favorablemente, pero también aumenta el efecto de una variación desfavorable sobre el conjunto de la cartera.

La diversificación consiste en repartir los recursos entre alternativas distintas para reducir esa dependencia. No elimina el riesgo, pero evita que el resultado global dependa por completo de una sola alternativa. La exposición permite cuantificar esa dependencia al indicar qué parte de los recursos está vinculada a un activo, una categoría o una plataforma. Ambas nociones permiten comparar distintas distribuciones de una misma cartera.

La composición de la cartera no es necesariamente permanente. El reequilibrio consiste en revisar la distribución de los recursos y ajustarla cuando deja de responder al nivel de riesgo previsto. Esta revisión puede deberse a cambios en el valor de los activos, a la aparición de nuevas alternativas o a cambios en la información disponible. Al evaluar un reequilibrio deben considerarse los costes de transacción, ya que comprar o vender para modificar la cartera reduce el resultado obtenido.

También interviene el horizonte temporal, que determina durante cuánto tiempo se prevé mantener una decisión antes de revisarla. Un horizonte más amplio puede admitir variaciones de precio durante más tiempo, mientras que uno breve requiere que los recursos estén disponibles antes.

Por tanto, la gestión de carteras combina la rentabilidad esperada, la concentración de los recursos, el plazo de las decisiones y los costes asociados a modificarlas.

2.6 Aplicaciones y trabajos relacionados

El aprendizaje automático se ha aplicado a problemas de análisis y decisión en los que los datos cambian con el tiempo. El aprendizaje supervisado permite estimar una variable a partir de observaciones históricas. El aprendizaje no supervisado permite descubrir agrupaciones o patrones en datos sin etiqueta. El aprendizaje por refuerzo permite estudiar decisiones encadenadas cuando una acción modifica las condiciones de las decisiones posteriores.

En la gestión automatizada de carteras, los métodos de aprendizaje por refuerzo se emplean para representar decisiones sucesivas de asignación, mantenimiento o modificación de recursos. La evaluación de estos métodos requiere conservar el orden temporal de los datos y reproducir las condiciones en las que se habría tomado cada decisión. También es necesario incorporar los costes de transacción, pues una estrategia que parece favorable antes de aplicarlos puede dejar de serlo después [17].

El aprendizaje por refuerzo multiagente se ha utilizado para estudiar situaciones en las que varios responsables de decisión comparten un mismo entorno. Estos responsables pueden cooperar, competir o combinar ambos comportamientos. La coordinación permite repartir una tarea compleja entre políticas distintas, aunque añade dificultades como la dependencia entre las decisiones de los agentes y la asignación del resultado colectivo a cada uno de ellos [14, 15].

Estos trabajos muestran que la utilidad de un método no debe valorarse solo por el resultado que obtiene en los datos empleados para entrenarlo. También debe comprobarse su comportamiento con datos no utilizados durante el entrenamiento, su sensibilidad a cambios en las condiciones del entorno y la claridad de los supuestos de evaluación. Estos criterios permiten comparar métodos de aprendizaje y de decisión de forma reproducible.

Contextualización del caso de estudio: activos digitales en Counter-Strike 2

Este capítulo concreta el marco general presentado en el Capítulo 2 mediante el caso de estudio de *Counter-Strike 2*, un videojuego multijugador en el que las personas usuarias pueden obtener y utilizar artículos digitales asociados a sus cuentas. Los artículos que se estudian se intercambian en Steam Community Market, un mercado secundario integrado en la plataforma Steam, y en BUFF, una plataforma externa que opera un mercado secundario. Este caso permite estudiar decisiones de compraventa cuando el artículo, el mercado secundario y las reglas de intercambio condicionan conjuntamente el resultado de una operación.

El interés del caso se encuentra en la fragmentación de precios entre mercados secundarios. Un mismo artículo puede publicarse a precios distintos en dos mercados, lo que permite plantear una compra en el mercado más barato y una venta posterior en el mercado con mayor precio. En un mercado bursátil líquido, las diferencias de precio de una misma acción suelen corregirse rápidamente. La existencia de plataformas separadas que administran estos mercados y aplican reglas de intercambio propias hace que *Counter-Strike 2* sea un caso de estudio adecuado para analizar si una diferencia de precios puede convertirse en un beneficio neto.

El caso de estudio se centra en artículos con precios registrados en los mercados secundarios de BUFF y Steam Community Market, cuyas publicaciones pueden verificarse como el mismo artículo. El capítulo describe los artículos, los mercados y las condiciones de intercambio que delimitan este análisis. No presenta todavía la arquitectura, las expresiones económicas ni el simulador, que se desarrollan en los capítulos posteriores.

3.1 Justificación, alcance y limitaciones

El motivo de seleccionar *Counter-Strike 2* es analizar decisiones de compraventa de activos digitales que persiguen generar un beneficio neto a partir de diferencias de precio entre mercados secundarios. Una vez comprobado que dos publicaciones se refieren exactamente al mismo artículo, una diferencia entre sus precios permite plantear una compra en el mercado más barato y una venta posterior en el mercado con mayor precio. El objetivo no es asumir que toda diferencia de precios es rentable, sino determinar cuáles merecen evaluarse como posibles operaciones.

Esta situación no es exclusiva de los activos digitales, pero presenta características distintas de un mercado bursátil líquido. Las unidades de una misma acción ordinaria son intercambiables entre sí y las diferencias de precio para el mismo instrumento suelen corregirse rápidamente. En cambio, los mercados secundarios de activos digitales se

encuentran administrados por plataformas separadas que publican información y aplican reglas de intercambio propias. Esta fragmentación permite observar diferencias de precio más visibles y convierte a *Counter-Strike 2* en un caso adecuado para el objetivo de este trabajo.

El análisis se limita a los artículos con precios registrados en BUFF y Steam Community Market cuyas publicaciones pueden verificarse como el mismo artículo. Esta delimitación permite estudiar una comparación concreta y trazable sin pretender representar todos los artículos de *Counter-Strike 2*, todas las plataformas ni todos los mecanismos de intercambio del videojuego.

3.2 Artículos y atributos de Counter-Strike 2

Los artículos de este caso de estudio pertenecen a las categorías cosméticas y coleccionables descritas en el Capítulo 2. Los artículos cosméticos no modifican las reglas de una partida, pero pueden tener precios diferentes según sus características y el volumen de intercambios existente. Los principales tipos de artículos del caso de estudio son:

- **Skins:** artículos que cambian el diseño visual de un arma. Se pueden equipar durante una partida, pero no alteran ninguna regla del juego.
- **Cuchillos:** artículos que sustituyen el aspecto del cuchillo básico equipado por el personaje. Su categoría y acabado hacen que sean especialmente relevantes como artículos coleccionables.
- **Gautes:** artículos que modifican el aspecto de las manos del personaje. Pueden combinarse visualmente con un cuchillo o con una *skin*.
- **Pegatinas:** elementos que se aplican a armas compatibles para personalizarlas. Su diseño y la colección de la que proceden pueden influir en su valor de mercado.
- **Cajas:** contenedores que pueden intercambiarse o abrirse. Al abrir una caja se consume y el usuario obtiene un artículo de la colección asociada.

El interés de estos artículos no depende solo de su utilidad dentro del videojuego. También intervienen la disponibilidad, la rareza, la colección a la que pertenecen y la demanda existente en los mercados. Por este motivo, dos artículos del mismo tipo pueden mantener precios muy diferentes. El caso de estudio se centra en los artículos que pueden identificarse y compararse entre los mercados secundarios seleccionados.

Para comparar publicaciones de dos mercados secundarios es necesario determinar si ambas se refieren exactamente al mismo artículo. El nombre base no siempre es suficiente. Por ejemplo, dos artículos denominados «AK-47 | Slate» pueden corresponder a condiciones de desgaste distintas y, por tanto, negociarse con precios distintos.

La identificación del artículo combina el nombre con los atributos que se indican a continuación:

- **Nivel de desgaste:** categoría que describe el desgaste visible del artículo, como *Factory New* o *Battle-Scarred*.
- **Float:** valor numérico asociado al desgaste. En general, un valor más próximo a cero indica un desgaste visual menor.
- **Rareza:** nivel que expresa la dificultad para obtener el artículo. Una rareza mayor no implica necesariamente un precio mayor.
- **StatTrak:** versión especial que incorpora un contador del número de eliminaciones logradas con el arma.

Estos atributos permiten construir un identificador para cada artículo y evitan comparar precios de activos digitales que solo parecen equivalentes. La normalización de este identificador se describe en el Capítulo 5, donde se explica cómo se preparan los datos antes de utilizarlos en los modelos y en la simulación.

3.3 Mercados secundarios y condiciones de intercambio

El caso de estudio combina dos mercados secundarios administrados por plataformas distintas con una fuente auxiliar de descubrimiento. Cada fuente aporta una parte diferente de la información necesaria para estudiar una posible operación:

- **SteamDT** se utiliza para localizar artículos candidatos que posteriormente se verifican en BUFF y Steam Community Market. No proporciona el precio de compra ni el precio de venta de la operación simulada, ni establece sus condiciones económicas [18].
- **Steam Community Market** es el mercado secundario integrado en la plataforma Steam. Publica ofertas de venta y órdenes de compra de artículos asociados a las cuentas de las personas usuarias. En la operación de compraventa, el precio de una oferta de venta observada en este mercado se utiliza como referencia del precio de venta simulado. También aporta información histórica y de actividad de intercambio [3].
- **BUFF** es una plataforma externa que opera un mercado secundario de artículos transferibles. En la operación de compraventa, el precio de una oferta de venta observada en BUFF se utiliza como referencia del precio de compra simulado. Los precios se conservan en CNY y con su conversión a EUR para permitir la comparación, junto con la actividad de intercambio y las órdenes de compra [7].

La operación de compraventa compara una oferta de venta en BUFF con una oferta de venta en Steam Community Market para el mismo artículo. Las órdenes de compra

describen una intención distinta, ya que una persona ofrece dinero hasta un importe máximo para adquirir un artículo. Por ello, se emplean como información complementaria y no como el precio principal de la operación simulada.

Para determinar si esa diferencia de precios puede estudiarse como una operación, ambos precios deben expresarse en una moneda común y el precio de venta debe ajustarse con las comisiones aplicables. Una diferencia entre precios antes de esos ajustes no permite determinar por sí sola el resultado de una operación.

El volumen de intercambios, es decir, la cantidad de intercambios registrados durante un periodo, aporta información sobre la actividad observada para un artículo en cada mercado secundario. Cuando el volumen es bajo, puede ser más difícil completar una venta en el plazo previsto. Asimismo, el periodo de bloqueo de intercambio (*trade hold*) puede impedir temporalmente vender o transferir un artículo adquirido. Durante ese periodo, el importe dedicado a la compra constituye capital bloqueado, es decir, saldo que no puede reutilizarse mientras la posición permanezca abierta o bloqueada. Estas condiciones de intercambio afectan tanto a la disponibilidad del capital como al momento en que una posición puede cerrarse.

La comparación de precios se utiliza para seleccionar candidatos que pueden estudiarse en la simulación. No supone que el artículo pueda comprarse o venderse al precio publicado, ni garantiza que ese precio se mantenga hasta completar el intercambio. El Capítulo 5 detalla la captura y normalización de los datos, mientras que los Capítulos 4 y 5 presentan las expresiones económicas y las reglas del simulador.

Arquitectura multiagente propuesta

En este capítulo se describe la arquitectura planteada para convertir datos de mercado en decisiones simuladas. La arquitectura establece una coordinación cooperativa entre roles diferenciados que comparten el objetivo de evaluar la cartera simulada. Se presentan sus capas, el flujo de información, el entorno de cartera y la distribución de responsabilidades entre los agentes Scout, Trader y Portfolio.

4.1 Formulación técnica del sistema

A partir del planteamiento descrito en el Capítulo 1, la propuesta traduce el caso de uso a una tarea de decisión secuencial sobre una cartera simulada de activos digitales, siguiendo la formulación general de los problemas de aprendizaje por refuerzo [11]. En cada instante, el sistema recibe observaciones de mercado, estado de cartera y señales derivadas. A partir de esa información decide si conviene comprar, vender, mantener en observación o rechazar una oportunidad.

La formulación técnica incorpora las fricciones que condicionan cada acción: beneficio neto después de comisiones, cambio de moneda, volumen de intercambios observado, capital ya bloqueado, tiempo de desbloqueo por *trade hold* y exposición acumulada a un mismo activo o plataforma. Con estos elementos, el objetivo del sistema es maximizar el valor total de la cartera simulada bajo restricciones de riesgo y fricción de mercado.

La arquitectura propuesta separa esta formulación en dos niveles. El primer nivel observa y transforma el mercado: extrae datos, los normaliza, los guarda y calcula señales supervisadas. El segundo nivel toma decisiones dentro de un simulador: recibe observaciones ya preparadas, emite acciones y recibe recompensas según el comportamiento de la cartera. Esta separación permite aislar la adquisición de datos, la limpieza, la predicción y el control de riesgo como responsabilidades diferenciadas.

4.2 Visión general del sistema

La arquitectura se organiza en cinco capas: adquisición, persistencia, predicción, decisión multiagente y simulación. Cada capa tiene una responsabilidad concreta y se comunica con la siguiente mediante datos normalizados. Esta decisión de diseño reduce el acoplamiento y permite validar cada parte por separado, una práctica habitual en entornos de trading reproducibles y backtesting [17].

La decisión final siempre queda registrada como simulada. Esto permite evaluar el sistema con métricas de cartera sin exponer capital durante la investigación. La arquitectura puede generar recomendaciones, episodios y métricas, pero no envía órdenes a Steam, BUFF ni ninguna otra plataforma.

1. Adquisición	Steam, BUFF, SteamDT, OCR e importaciones manuales capturan observaciones.
2. Persistencia	Postgres/Supabase guarda artículos, históricos, snapshots, predicciones y decisiones simuladas.
3. Predicción	El modelo supervisado genera probabilidades calibradas y señales de mercado.
4. Decisión MARL	Scout, Trader y Portfolio deciden dentro del entorno PettingZoo/RLlib.
5. Simulación	La cartera aplica comisiones, divisas, <i>trade hold</i> , riesgo y métricas.

Figura 4.1: Capas del sistema.

4.3 Flujo operativo

El flujo operativo comienza con artículos localizados mediante una búsqueda de mercado o una fuente de descubrimiento. Los agentes extractores consultan las fuentes disponibles, capturan precios, buy orders, volumen y metadatos, y construyen observaciones normalizadas. Después, la persistencia conserva el histórico en formato auditable para que el dataset pueda reconstruirse sin depender de una sesión concreta de scraping.

Sobre ese histórico se construyen características de mercado. El modelo supervisado produce una probabilidad calibrada de oportunidad rentable en un horizonte determinado. Esa probabilidad no se interpreta como una orden de compra, sino como una característica adicional que los agentes MARL pueden utilizar. Finalmente, el entorno multiagente genera una observación para cada política, recoge sus acciones y aplica las reglas del simulador.

Algoritmo 1 Ciclo de decisión

- 1: Recoger observaciones recientes del mercado.
 - 2: Normalizar precios, moneda, volumen de intercambios y metadatos de plataforma.
 - 3: Actualizar el estado de cartera y las variables de riesgo.
 - 4: Calcular señales supervisadas disponibles para el episodio.
 - 5: Generar una observación local para Scout, Trader y Portfolio.
 - 6: Obtener acciones de los agentes y aplicar restricciones de riesgo.
 - 7: Simular la decisión final sobre la cartera.
 - 8: Calcular recompensa, métricas y trazabilidad del episodio.
-

4.3.1 Clasificación de oportunidades

Después de recopilar los precios y las condiciones de un activo digital, el sistema clasifica la posible operación en uno de tres estados. Esta clasificación no sustituye la decisión de la persona usuaria, sino que organiza las posibles operaciones que conviene atender antes.

Revisar La información disponible y el resultado económico estimado justifican una comprobación manual.

Observar Faltan datos o conviene esperar una nueva captura antes de evaluar la posible operación.

Bloquear Los costes, el riesgo o las restricciones de cartera impiden continuar con la posible operación.

4.4 Definición del entorno

El entorno representa una cartera simulada que evoluciona a partir de datos históricos. Cada episodio contiene una secuencia de observaciones de mercado y un estado interno formado por saldo disponible, posiciones abiertas, posiciones bloqueadas, fechas de desbloqueo y métricas de exposición.

4.4.1 Formulación como MDP

El problema se formula como un proceso de decisión de Markov, $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, donde $\mathcal{S}$ representa los estados de la cartera simulada, $\mathcal{A}$ las acciones posibles, $\mathcal{P}$ la transición entre estados, $\mathcal{R}$ la recompensa y γ el factor de descuento de las recompensas futuras.

El objetivo del aprendizaje por refuerzo es obtener la política π^* que maximiza la recompensa acumulada esperada durante un episodio:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \right]. \quad (4.1)$$

En la Ecuación 4.1, π es la regla con la que el agente selecciona acciones, r_{t+1} es la recompensa obtenida tras cada acción y $\mathbb{E}_{\pi}$ representa el promedio sobre las posibles evoluciones del episodio al seguir esa política. El factor γ da menos peso a las recompensas más lejanas. En este trabajo, la recompensa se define a partir de la evolución de la cartera simulada y penaliza costes, riesgo, un volumen de intercambios bajo y capital bloqueado.

En este trabajo, el estado incluye la información del mercado, el saldo disponible, las posiciones abiertas y bloqueadas, las fechas de desbloqueo y la exposición acumulada. Las acciones permiten comprar, vender, mantener u observar una oportunidad, siempre sujetas a las restricciones de riesgo. La transición actualiza el saldo, las posiciones, el capital bloqueado y la exposición después de cada acción. La recompensa evalúa cómo cambia el valor neto de la cartera simulada, considerando beneficio, costes, volumen de intercambios y riesgo.

La exploración y el entrenamiento se realizan únicamente dentro del simulador y con datos históricos. Las acciones exploratorias modifican una cartera simulada, no una cuenta real ni los activos digitales asociados a ella. De este modo, el sistema puede probar decisiones de compra, venta o mantenimiento y observar sus consecuencias — comisiones, *trade hold*, volumen de intercambios y riesgo— sin utilizar capital real.

4.4.2 Estado y observaciones

Las observaciones combinan variables de mercado y variables de cartera. Entre las variables de mercado se incluyen precios de Steam y BUFF normalizados a EUR, precios

originales en su moneda nativa, volumen de intercambios, spread bruto, spread neto, órdenes de compra (*buy orders*), antigüedad del activo virtual y señales temporales como retornos, medias móviles, RSI o desviaciones de ventas. Entre las variables de cartera se incluyen efectivo disponible, capital bloqueado, posiciones abiertas, exposición por activo y estado de desbloqueo.

La salida del modelo supervisado se incorpora como una probabilidad calibrada de que una oportunidad mantenga un spread rentable en el horizonte definido. Esta señal permite aprovechar aprendizaje supervisado sin convertirlo en una decisión rígida. Si la probabilidad es alta pero la cartera está sobreexpuesta o el capital está bloqueado, el entorno puede rechazar o posponer la operación.

4.4.3 Espacio de acciones

El espacio de acciones se mantiene deliberadamente simple. Las acciones principales son comprar, vender, mantener y observar. En la parte de compra o venta puede añadirse un tamaño de posición discreto para limitar el número de combinaciones. Esta simplificación permite entrenar y depurar el entorno antes de introducir acciones continuas o reglas de asignación más complejas.

En la versión implementada, Scout puede ignorar o marcar una oportunidad. Trader puede mantener, solicitar la compra de una unidad o solicitar la venta de una posición desbloqueada del artículo observado. Portfolio rechaza o aprueba la operación según las restricciones de cartera. Una compra requiere el acuerdo de los tres roles. Una venta requiere la solicitud de Trader, la aprobación de Portfolio, una posición abierta del mismo artículo y el fin del *trade hold*.

Algoritmo 2 Decisión cooperativa

Entrada: Candidato c_t , estado de cartera s_t y observaciones locales

Salida: Transición de cartera y recompensa compartida R_t

```

1:  $a_t^S \leftarrow \pi_S(o_t^S)$ 
2:  $a_t^T \leftarrow \pi_T(o_t^T, m_t^T)$ 
3:  $a_t^P \leftarrow \pi_P(o_t^P, m_t^P)$ 
4: if  $a_t^S = \text{marcar}$  and  $a_t^T = \text{comprar}$  and  $a_t^P = \text{aprobar}$  and  $c_t$  cumple los límites
   then
5:   Abrir una posición simulada y bloquear su capital
6: else if  $a_t^T = \text{vender}$  and  $a_t^P = \text{aprobar}$  and existe una posición desbloqueada de  $c_t$ 
   then
7:   Cerrar una posición simulada y abonar su importe neto
8: else
9:   Mantener el saldo y registrar el motivo de no ejecución
10: end if
11: Actualizar posiciones liquidables, saldo, exposición y drawdown
12: Calcular  $R_t$  y devolver la misma recompensa a los tres agentes

```

Las tres primeras líneas recogen la acción propuesta por cada política: Scout marca o ignora, Trader mantiene, compra o solicita una venta y Portfolio aprueba o rechaza el riesgo. Las variables m_t^T y m_t^P incorporan las métricas de mercado y cartera que

necesita cada rol para decidir. En una venta, Scout no necesita emitir una marca nueva porque la posición ya forma parte de la cartera.

4.4.4 Transición del entorno

Cuando se ejecuta una compra simulada, el saldo disponible disminuye y se abre una posición. Esa posición queda bloqueada durante el periodo de *trade hold*. Mientras está bloqueada no puede venderse, pero sí afecta a la exposición y al capital inmovilizado. Cuando se ejecuta una venta simulada, el simulador calcula el ingreso neto aplicando comisiones, conversión de moneda y reglas de valoración. El estado de cartera se actualiza después de cada paso.

4.5 Agentes del sistema

La arquitectura distingue entre agentes extractores y agentes MARL. Los extractores no se entrenan mediante aprendizaje por refuerzo y no deciden operaciones. Su trabajo es adquirir datos desde Steam, BUFF, SteamDT, OCR o CSV/API y transformarlos en observaciones persistidas. Los agentes MARL, en cambio, operan dentro del entorno de simulación.

Agente	Función
<i>Adquisición de datos</i>	
Steam y BUFF	Recogen precios, histórico, buy orders y volumen.
SteamDT	Descubre candidatos para priorizar la captura.
OCR y CSV	Convierten capturas y archivos en observaciones.
<i>Decisión multiagente</i>	
Scout	Filtra oportunidades por margen, volumen de intercambios y señal.
Trader	Propone comprar, vender, mantener u observar.
Portfolio	Valida exposición, capital bloqueado y riesgo.

Tabla 4.1: Roles del sistema.

4.5.1 Scout

Scout se encarga de detectar oportunidades. Su observación está orientada a señales de precio, spread, volumen de intercambios y probabilidad supervisada. Su papel no es decidir toda la operación, sino marcar qué activos parecen merecer atención dentro del episodio.

4.5.2 Trader

Trader decide la acción operativa principal: comprar, vender, mantener u observar. Para ello considera la señal de Scout, el estado de mercado, el posible beneficio neto y el tamaño de posición permitido. Su responsabilidad está más cerca de la ejecución de una oportunidad concreta.

4.5.3 Portfolio

Portfolio controla la coherencia global de la cartera. Su observación incorpora saldo, capital bloqueado, posiciones abiertas, exposición máxima y restricciones de riesgo. Este agente puede penalizar o bloquear decisiones que parezcan rentables de forma aislada pero que deterioren el perfil global de la cartera.

4.6 Función de recompensa

La recompensa debe reflejar el objetivo del sistema: mejorar el valor neto de la cartera sin ignorar riesgo ni fricción. Una recompensa basada solo en comprar barato o en detectar subida de precio sería incompleta, porque no tendría en cuenta si la operación puede cerrarse, si el capital queda bloqueado o si el spread desaparece antes de poder vender.

4.6.1 Recompensa individual

Las señales individuales pueden ayudar a entrenar comportamientos especializados. Scout puede recibir recompensa por identificar oportunidades que más adelante resultan útiles. Trader puede recibirla por ejecutar acciones con beneficio neto y Portfolio por evitar sobreexposición, operaciones ilíquidas o capital bloqueado excesivo. Estas recompensas no sustituyen al objetivo global, pero pueden guiar el aprendizaje de cada rol.

Como ampliación de la arquitectura, la recompensa de cada rol puede combinar una parte compartida, vinculada a la evolución de la cartera, y una parte individual, vinculada a su responsabilidad concreta:

$$r_t^i = \lambda r_t^{\text{cartera}} + (1 - \lambda) r_{t,\text{rol}}^i, \quad 0 \leq \lambda \leq 1. \quad (4.2)$$

En la Ecuación 4.2, r_t^{cartera} es la recompensa común, $r_{t,\text{rol}}^i$ mide la contribución del agente i y λ determina el peso de cada parte. La parte compartida debe tener un peso mayor para que todos los agentes mantengan como prioridad el resultado global de la cartera. Las ejecuciones descritas en este trabajo utilizan la recompensa compartida de la sección siguiente. La recompensa individual queda definida como una extensión que deberá evaluarse con datos suficientes.

4.6.2 Recompensa cooperativa

La recompensa cooperativa se vincula al valor total de la cartera simulada. Considera beneficio realizado, retorno de la operación ejecutada, inactividad ante una oportunidad permitida, restricciones de riesgo, *drawdown*, capital bloqueado y volatilidad. La expresión implementada es:

$$r_t = \frac{\Delta \Pi_t}{100} + \rho_t \mathbb{I}_{\text{operacion}} - 0,01 \mathbb{I}_{\text{oportunidad ignorada}} - 0,05 v_t - 0,50 DD_t - 0,05 B_t - 0,10 \sigma_t \mathbb{I}_{\text{compra}}.$$

Aquí, $\Delta \Pi_t$ es el cambio en beneficio realizado en euros, ρ_t es el retorno de la compra o venta ejecutada, v_t cuenta las restricciones incumplidas, DD_t es el *drawdown* relativo,

B_t es la fracción de cartera bloqueada y σ_t representa la volatilidad disponible para el candidato. El divisor 100 evita que una ganancia expresada en euros domine al resto de términos. La misma recompensa se asigna a Scout, Trader y Portfolio, es decir, $r_t^S = r_t^T = r_t^P = r_t$, para que los tres roles persigan el estado global de la cartera.

4.7 Restricciones de riesgo

Las restricciones de riesgo se introducen antes de escalar el entrenamiento. Las más importantes son: no usar capital inexistente, limitar exposición por activo o plataforma, rechazar operaciones con un volumen de intercambios insuficiente, respetar el *trade hold*, controlar el capital bloqueado y separar claramente simulación de operación. Estas restricciones de riesgo son sencillas, pero evitan que los agentes aprendan políticas imposibles de ejecutar en el mercado.

La salida del sistema debe ser siempre auditable. Cada predicción, acción o decisión simulada debe poder relacionarse con su fuente de datos, versión de modelo, episodio y configuración de simulación. Esta trazabilidad es necesaria para interpretar resultados y para distinguir entre error de datos, error de modelo y mala política de decisión.

Configuración del entorno y datos

Este capítulo explica qué datos utiliza el sistema, cómo se obtienen y cómo se preparan antes de emplearlos en la simulación o en los modelos. También presenta las reglas económicas y las variables que describen el estado de una cartera.

5.1 Fuentes de datos

El sistema trabaja con varias fuentes porque el mercado de CS2 está fragmentado. Ninguna fuente aislada contiene toda la información necesaria para decidir. Steam aporta precio, histórico y señales del mercado principal. BUFF permite comparar oportunidades y buy orders. SteamDT ayuda a descubrir candidatos. El OCR permite capturar datos cuando no existe una API cómoda. Los CSV o importaciones manuales sirven para integrar muestras controladas o material histórico.

La capa de adquisición se ha desarrollado en Python 3.11. HTTPX se emplea cuando una fuente admite una consulta HTTP estable y Playwright cuando exige navegación o renderizado de página. Las observaciones se guardan en PostgreSQL, desplegado mediante Supabase, y el acceso se organiza con SQLAlchemy y AsyncPG. Así, cada dato conserva su fuente, instante de captura y formato original antes de incorporarse al análisis.

5.1.1 Mercado de Steam

Steam actúa como una de las fuentes principales de precio e histórico. En el proyecto se capturan artículos identificados por sus atributos, precios actuales, moneda, buy orders y puntos históricos cuando están disponibles. La información se normaliza para que pueda combinarse con otras plataformas y para evitar que las decisiones dependan de nombres o formatos propios de una fuente.

En la práctica, Steam no se usa solo como una tabla de precios. También aporta señales de volumen de intercambios y de comportamiento temporal. El histórico permite construir retornos, medias móviles, volatilidad y desviaciones frente a ventas recientes. Estas variables son necesarias para que el modelo no mire únicamente una fotografía puntual del mercado.

5.1.2 BUFF y mercados externos

BUFF se utiliza como mercado externo de comparación. Sus precios se reciben principalmente en CNY, por lo que el sistema conserva tanto el valor original como su conversión a EUR. Esta doble representación permite auditar el dato original y, al mismo tiempo, entrenar modelos sobre una moneda canónica. Las buy orders de BUFF son

especialmente útiles porque ayudan a estimar si una salida tiene demanda o si el precio observado es poco ejecutable.

SteamDT se utiliza como apoyo para descubrir candidatos y estrategias de búsqueda, no como fuente única de decisión. Su función dentro del sistema es reducir el espacio de búsqueda inicial y ayudar a priorizar qué artículos merece la pena observar con más profundidad.

Tras las primeras pruebas, BUFF se separa del entrenamiento principal. El histórico masivo se apoya en Steam, mientras que BUFF se usa como precio vivo de entrada cuando se evalúa un flip concreto hacia Steam. De este modo el modelo no intenta predecir el precio de BUFF ni depende de scraping recurrente de BUFF para aprender patrones temporales.

5.1.3 OCR e importación manual

El OCR se incorpora como vía secundaria de adquisición. Su objetivo es convertir capturas, tablas o interfaces no estructuradas en observaciones equivalentes a las obtenidas por scraping o API. El flujo incluye preprocesado de imagen, reconocimiento de texto, parsing estructurado, validación y persistencia. Aunque es más frágil que una fuente estructurada, resulta útil cuando una plataforma no expone datos cómodamente o cuando se quiere incorporar material histórico procedente de capturas.

La implementación actual se centra en tablas y mensajes de mercado, que son las capturas con una estructura suficientemente regular para poder validarse. OpenCV amplía la imagen por un factor dos, la convierte a escala de grises, mejora el contraste local con CLAHE, reduce ruido y aplica una binarización adaptativa. CLAHE divide la imagen en zonas, ajusta el contraste de cada una y limita ese refuerzo para no amplificar el ruido. Tesseract devuelve las palabras y una confianza por palabra. La confianza agregada utilizada por el pipeline es la media de las n palabras reconocidas con confianza válida:

$$c_{\text{OCR}} = \frac{1}{100n} \sum_{j=1}^n c_j.$$

El parser normaliza espacios y separadores decimales, identifica precio, moneda, calidad, plataforma y volumen cuando aparecen en la línea. Solo se convierten en observaciones las extracciones con $c_{\text{OCR}} \geq 0,5$ y con un precio positivo menor de un millón de unidades monetarias. Esta regla evita que una captura dudosa entre silenciosamente al histórico.

Algoritmo 3 Extracción OCR.

Entrada: Captura I , confianza mínima τ y contexto de fuente

Salida: Observaciones normalizadas o rechazo trazable

```
1: Cargar  $I$  y aplicar escala, contraste, reducción de ruido y binarización
2: Ejecutar Tesseract y calcular  $c_{OCR}$ 
3: if  $c_{OCR} < \tau$  then
4:   Rechazar la captura y conservar su referencia para revisión
5: else
6:   for all líneas reconocidas do
7:     Extraer artículo, calidad, plataforma, precio, moneda y volumen
8:     if precio y moneda son válidos then
9:       Construir identificador canónico y observación con la confianza obtenida
10:    end if
11:  end for
12: end if
```

Etapa	Salida	Control aplicado
Preprocesado	Imagen binaria ampliada	CLAHE, reducción de ruido y umbral adaptativo.
Reconocimiento	Texto y confianza por palabra	Media de confianzas válidas de Tesseract.
Parsing	Campos de mercado normalizados	Precio, moneda, calidad, plataforma y volumen.
Validación	Observación o rechazo	Confianza mínima, precio positivo y rango máximo.

Tabla 5.1: Controles del pipeline OCR.

Las importaciones manuales y CSV cumplen una función complementaria. Permiten introducir muestras controladas, fixtures o históricos parciales sin depender de una sesión de navegador. Esta vía ha sido útil para validar parsers, modelos de persistencia y reglas económicas antes de automatizar el flujo completo.

5.2 Adquisición y persistencia

Esta sección describe el recorrido de un dato desde su captura hasta su almacenamiento. El objetivo es conservar observaciones comparables y trazables, de modo que puedan utilizarse más adelante en la preparación de variables y en los experimentos.

5.2.1 Pipeline de adquisición

La adquisición se organiza como una capa de agentes extractores. Estos procesos pueden ejecutarse como comandos CLI, jobs o servicios programados. Su función es capturar datos, normalizarlos, validar errores y guardarlos con trazabilidad. El flujo incluye scraping, refresco de históricos, OCR, importación manual y un bucle operativo local que puede ejecutar scraping y actualización periódica.

La adquisición periódica contempla cookies o sesiones, delays configurables, límites de concurrencia, reintentos, deduplicación y registro de anomalías. Esta parte es importante porque las fuentes no siempre son estables: pueden cambiar el formato, limitar el ritmo de acceso o devolver información incompleta.

5.2.2 Modelo de persistencia

La persistencia se apoya en Supabase/Postgres. La decisión de diseño es tratar la base de datos como fuente de verdad y mantener el histórico de forma append-only siempre que sea posible. Las tablas operativas principales son `market_items` y `market_history_points`. La primera mantiene una fila por artículo identificado por sus atributos y estado actual por plataforma. La segunda guarda series temporales en formato largo por artículo, plataforma, fecha y métrica.

También se define un esquema canónico más amplio con entidades como `assets`, `platforms`, `market_observations`, `predictions`, `investment_decisions` y `simulated_positions`. Esta separación permite evolucionar desde el esquema operativo actual hacia una representación más trazable y auditable.

Entidad	Uso dentro del TFM
<code>market_items</code>	Estado actual de cada artículo identificado por sus atributos, por plataforma.
<code>market_history_points</code>	Serie temporal en formato largo para precios, buy orders y métricas.
<code>predictions</code>	Salidas versionadas del modelo supervisado.
<code>investment_decisions</code>	Decisiones simuladas y razones asociadas.
<code>simulated_positions</code>	Posiciones abiertas, bloqueadas o cerradas por el simulador.

Tabla 5.2: Modelo de datos.

5.3 Preparación de características

Los datos normalizados se preparan con Pandas y NumPy. PyArrow permite conservar los datasets derivados en un formato reproducible. Estas herramientas se usan aquí para construir variables temporales y cortes de evaluación, no como parte del marco teórico.

Antes de entrenar un modelo o simular una cartera, las observaciones se transforman en variables que resumen precios, volumen de intercambios, evolución temporal y estado de la cartera. A continuación se explican las variables empleadas y la construcción temporal de las etiquetas.

5.3.1 Variables de mercado

Las características de mercado incluyen precios Steam y BUFF normalizados, precios originales, spread bruto, spread neto, buy orders, volumen de intercambios, número de listings, antigüedad del activo virtual y señales temporales. En la fase supervisada se han explorado variables de momentum y reversión como retornos a 1, 3 y 7 días, distancia a

medias móviles, RSI, volatilidad y desviaciones de ventas. Esta combinación de señales temporales y restricciones operativas sigue la misma lógica que los entornos de trading con control de fricciones [17].

La exploración inicial detectó que algunas señales temporales aportan información moderada, mientras que ciertas variables categóricas de alta cardinalidad, como identificadores concretos de skins, deben tratarse con cuidado para evitar sobreajuste. Por ello, el entrenamiento separa variables de evaluación y variables de entrenamiento, y permite realizar ablations sin reconstruir todo el dataset.

5.3.2 Dataset temporal de trading

Para cada observación de un artículo en el día t , el constructor busca la primera salida disponible a partir de $t + h$, donde $h = 8$ días representa el bloqueo aplicado por el simulador. La etiqueta no representa una subida de precio aislada, sino una operación que supera los umbrales económicos tras aplicar comisiones y conversión:

$$y_t = 1 \iff \pi_{t+h} \geq \pi_{\text{mín}} \wedge \rho_{t+h} \geq \rho_{\text{mín}}.$$

Los conjuntos se separan por fecha y se elimina una franja de purga antes de cada frontera para que una salida futura no coincida con el entrenamiento de un periodo anterior.

Algoritmo 4 Dataset temporal.

Entrada: Históricos Steam/BUFF, horizonte h , fechas de corte y purga g

Salida: Conjuntos de entrenamiento, validación y prueba trazables

- 1: **for all** observaciones (i, t) ordenadas por artículo y fecha **do**
 - 2: Calcular variables disponibles hasta t
 - 3: Buscar la primera salida del artículo i a partir de $t + h$
 - 4: **if** la salida es válida **then**
 - 5: Calcular π_{t+h} , ρ_{t+h} y la etiqueta y_t
 - 6: Conservar (i, t) , sus variables y la fecha de salida
 - 7: **end if**
 - 8: **end for**
 - 9: Excluir g días antes de cada frontera temporal
 - 10: Separar por fecha y guardar filas, artículos y rango de cada conjunto
-

5.3.3 Variables de cartera

Las variables de cartera proceden del simulador. Incluyen saldo disponible, valor de posiciones abiertas, capital bloqueado por *trade hold*, fecha de desbloqueo, exposición por activo, exposición por plataforma, beneficio realizado y drawdown. Estas variables son necesarias para que el agente Portfolio no decida únicamente por oportunidad local, sino por estado global de la cartera, coherentemente con la evaluación conjunta de rentabilidad y riesgo [16].

5.4 Reglas económicas migradas

El simulador formaliza las reglas económicas identificadas en el procedimiento manual de seguimiento. Incluye conversión EUR/CNY, comisiones, beneficio neto, rentabilidad porcentual, fecha de desbloqueo y capital bloqueado.

Regla	Uso en la simulación
Conversión EUR/CNY	Normaliza precios de Steam y BUFF para compararlos en una moneda común.
Fee BUFF 0,975	Estima el ingreso neto al vender en BUFF.
Fee Steam 0,87	Representa la comisión de mercado en ventas Steam.
Cash-out conservador	Permite simular pérdida adicional al convertir saldo Steam en saldo externo.
<i>Trade hold</i>	Bloquea posiciones durante un horizonte conservador de ocho días.
Beneficio neto	Calcula diferencia entre ingreso neto de venta y coste de entrada.

Tabla 5.3: Reglas de simulación.

5.5 Simulador de cartera

El simulador se ha implementado con módulos propios, ya que debe representar reglas específicas del dominio, como comisiones, conversión de moneda, capital bloqueado y *trade hold*. Esta capa recibe observaciones ya normalizadas y produce transiciones de cartera que pueden evaluarse de forma repetible.

El simulador representa las consecuencias de una decisión sobre el efectivo disponible, las posiciones abiertas y el riesgo. Sus reglas permiten comparar alternativas con las mismas comisiones, bloqueos y límites de exposición.

5.5.1 Operaciones permitidas

Las operaciones básicas son comprar, vender y mantener. Una compra abre una posición con precio de compra, cantidad, plataforma y fecha de bloqueo. Una venta cierra una posición si está disponible y calcula el beneficio realizado. Mantener no modifica la posición, pero puede afectar al rendimiento si el mercado se mueve o si el capital sigue inmovilizado.

5.5.2 Comisiones y valoración

El procedimiento manual aportó reglas de referencia que se han convertido en hipótesis de simulación. BUFF se modela con un factor neto de venta de 0,975. Steam usa una comisión de mercado del 0,87 y, en escenarios conservadores de salida a banco, puede aplicarse una pérdida adicional del 20 %. La conversión inicial simplificada usa $1 \text{ EUR} = 8 \text{ CNY}$, aunque queda pendiente versionar tipos de cambio por fecha si se incorporan fuentes externas.

El *trade hold* se modela de forma conservadora como ocho días desde la compra, equivalente a siete días más una ventana de desbloqueo. Esta simplificación evita declarar vendible una posición demasiado pronto.

Implementación y operación

Este capítulo traslada la propuesta a componentes de software. Describe cómo se conectan los flujos de adquisición, persistencia y consulta local para mantener la trazabilidad de las oportunidades. La estructura detallada del código se incluye en el Anexo A.1.

6.1 Organización del sistema

La implementación separa las responsabilidades de adquisición, persistencia, simulación, predicción y operación local. Esta separación permite ejecutar un scraper, repetir un entrenamiento o consultar el panel sin mezclar esas tareas con las reglas económicas y los contratos de datos.

La predicción tabular se ejecuta con scikit-learn y los modelos entrenados se conservan con Joblib. El entorno multiagente se implementa con PettingZoo y se conecta con Ray/RLlib; Gymnasium y PyTorch forman parte de ese ecosistema de entrenamiento. Las pruebas unitarias y de integración se ejecutan con Pytest y pytest-asyncio, mientras que Ruff y Mypy se emplean para revisar estilo y tipos. Cada herramienta se utiliza en el componente que corresponde, de modo que el código no sustituye a la explicación metodológica de los capítulos anteriores.

Las capas se comunican mediante contratos explícitos. Por ejemplo, un extractor no entrega directamente una recomendación. Entrega una observación con artículo, plataforma, fecha, precio, moneda, volumen, origen y calidad. La persistencia conserva esta información y los módulos de datos construyen después las variables necesarias para cada experimento. Esta secuencia facilita localizar un error sin tener que rehacer todo el flujo.

6.2 Flujo operativo de adquisición

Esta sección explica cómo se ejecuta la recogida de datos y cómo se mantiene su trazabilidad. Se describen las sesiones necesarias para acceder a las fuentes, el proceso de scraping y el almacenamiento incremental de las observaciones.

6.2.1 Sesiones, scraping y trazabilidad

Las fuentes que necesitan navegación se ejecutan con Playwright en un navegador visible cuando es necesario iniciar sesión. La sesión no se guarda en el repositorio ni en el archivo de configuración. Se almacena localmente como estado de navegador y se renueva desde la pantalla de sesiones. Esto evita que las credenciales formen parte de los experimentos o de los commits.

Cada trabajo de scraping registra inicio, conectores utilizados, reintentos, progreso, fichero de log y resultado final. Los límites de concurrencia, el tamaño de lote y el tiempo de espera son configurables. El objetivo no es ejecutar el mayor número de peticiones posible, sino mantener un proceso que pueda detenerse, reanudarse y auditarse si una fuente cambia su interfaz.

Algoritmo 5 Flujo de scraping.

Entrada: Artículos localizados, conectores activos y configuración de trabajo

Salida: Observaciones persistidas y estado de ejecución

```
1: Crear identificador de trabajo y registrar el inicio
2: for all lotes de candidatos do
3:   for all conectores habilitados do
4:     Consultar la fuente con sesión, límite y espera configurados
5:     Normalizar precio, moneda, disponibilidad y fecha de observación
6:     Validar el resultado y registrar anomalías o reintentos
7:   end for
8:   Persistir observaciones válidas y actualizar el progreso
9: end for
10: Registrar resultado, cobertura y referencia del log
```

6.2.2 Persistencia incremental

El modelo operativo almacena el estado actual en `market_items` y las series en `market_history_points`. Guardar el histórico en formato largo permite incorporar nuevas métricas sin alterar el esquema de una tabla ancha. Una observación incluye el origen y la fecha, por lo que una variación de precio puede relacionarse con la fuente y con la ejecución que la recogió.

La deduplicación se realiza antes de persistir y las actualizaciones de estado se comprueban con pruebas de integración. En la validación ejecutada se insertó un snapshot, se actualizó una cotización y se guardó una señal vinculada a un artículo. Cada prueba revierte su transacción al terminar, de modo que las comprobaciones no contaminan el histórico de experimentación.

6.3 Consola de consulta

La consola local convierte el estado técnico en una interfaz de revisión. Su pantalla principal separa la bandeja de oportunidades del detalle de la oportunidad seleccionada. La bandeja concentra el nombre del artículo, la ruta, el margen y los precios Steam y BUFF. El detalle explica qué señal la ha producido, qué datos faltan y qué restricciones de cartera intervienen.

El panel no ejecuta compras. Las acciones de abrir Steam o BUFF sirven para que la persona usuaria revise la oportunidad en su fuente. Los estados *revisar*, *observar* y *bloqueado* representan una decisión explicable, no una orden automática. Esta restricción es importante mientras la validación estadística y multiagente sigue en fase exploratoria.

Vista	Información que hace visible
Oportunidades	Ruta, precios, margen y estado de la señal.
Scraper	Progreso, conectores, logs y resultado.
Sesiones	Vigencia de Steam y BUFF.
Modelo	Versión, conjunto, métricas y carácter experimental.
Agentes y riesgo	Roles, capital bloqueado, límites y rechazos.

Tabla 6.1: Vistas de consola.

6.4 Reproducibilidad y configuración

La configuración se versiona mediante un fichero de ejemplo sin secretos. Para ejecutar la persistencia basta con definir `DATABASE_URL`. El resto de opciones habilita funciones concretas: ruta de Tesseract, token local del servidor de scraping, límites de concurrencia o parámetros de sesiones. Las claves de una base de datos y los estados de navegador quedan excluidos del control de versiones.

Los comandos de construcción de datasets y entrenamiento guardan el rango temporal, las columnas, los tamaños de cada partición, las tasas de clase, los hiperparámetros y las métricas obtenidas. Esta información permite distinguir dos situaciones que visualmente podrían parecer iguales: ejecutar un modelo con más datos y repetir exactamente una configuración anterior.

Algoritmo 6 Ejecución de experimento.

Entrada: Dataset versionado, fechas de corte, configuración y semilla

Salida: Informe, figura y modelo asociados a una ejecución

- 1: Construir entrenamiento, validación y prueba respetando el orden temporal
 - 2: Guardar el esquema de variables y las tasas de clase
 - 3: Entrenar los candidatos con la configuración declarada
 - 4: Elegir parámetros usando solo validación
 - 5: Evaluar una vez sobre prueba y guardar métricas
 - 6: Generar tabla o figura desde el informe guardado
-

Este procedimiento se aplica también a las pruebas automatizadas. La suite acompaña las modificaciones del sistema, de forma que las tablas y gráficas se apoyen en componentes que han pasado las comprobaciones de dominio, persistencia, OCR, scraping, simulación, MARL y panel web.

Experimentación y análisis

Este capítulo describe las pruebas realizadas sobre reglas económicas, datos, modelos supervisados, OCR y arquitectura multiagente. Cada resultado se interpreta según la evidencia disponible y se distinguen las comprobaciones funcionales de las comparaciones que requieren más datos.

7.1 Diseño experimental

La experimentación se ha desarrollado por fases. Primero, se han migrado reglas económicas y estructuras de datos. Después, se han construido datasets supervisados para estudiar señales de mercado. Más adelante, se ha creado un dataset de trading basado en precios Steam/BUFF y se ha implementado el entorno multiagente. Esta secuencia evita entrenar agentes MARL sobre un entorno que todavía no representa bien el problema económico. La Tabla 7.1 resume las ejecuciones que se desarrollan con detalle en las secciones siguientes.

Flujo	Comprobación ejecutada
Reglas económicas	Pruebas de conversión, comisiones, bloqueo temporal y límites.
Modelo de dirección	Entrenamiento, calibración y perfil de umbrales sobre histórico Steam.
Trading BUFF–Steam	Cortes temporales, ablaciones y comprobación de cobertura.
OCR	Parser, umbral de confianza y recorrido de imagen.
MARL	Dos baselines coordinados y smoke de PPO con tres políticas.

Tabla 7.1: Matriz experimental.

7.1.1 Escenarios de evaluación

Se han usado dos tipos de datasets. El primero predice dirección de mercado sobre histórico disponible y permite estudiar señales temporales de forma amplia. El segundo intenta modelar oportunidades de compraventa Steam/BUFF con precios normalizados, comisiones y horizonte temporal. Este segundo dataset es más cercano al objetivo del TFM, pero también es más exigente porque necesita histórico alineado entre plataformas.

El criterio de evaluación no es solo predictivo. Una señal puede tener buena precisión y aun así no ser operativa si aparece pocas veces, si depende de artículos concretos o

si no supera costes de transacción. Por eso las métricas del modelo supervisado se interpretan como apoyo a la decisión, no como resultado económico final.

7.1.2 Estrategias de comparación

La comparación experimental se plantea en tres niveles. El primero es un baseline simple o dummy para comprobar que los modelos aprenden algo más que la tasa base. El segundo compara modelos supervisados tabulares como regresión logística, random forest e histogram gradient boosting, familias ampliamente usadas para clasificación tabular [19, 20]. El tercero verifica que la arquitectura MARL recorre compra, bloqueo y venta con reglas deterministas, además de comprobar su conexión con PPO. Esta última parte valida coordinación, restricciones y trazabilidad, no la rentabilidad de una política entrenada.

Algoritmo 7 Entrenamiento PPO

Entrada: Entorno paralelo, políticas Scout, Trader y Portfolio, iteraciones y pasos

Salida: Checkpoint y métricas de la ejecución

- 1: Inicializar una política PPO por rol con observaciones vectoriales
 - 2: **for** cada iteración de entrenamiento **do**
 - 3: Ejecutar episodios y registrar $(o_t^i, a_t^i, R_t, m_t^i)$
 - 4: Estimar ventajas $\hat{A}_t$ a partir de las transiciones registradas
 - 5: Optimizar cada política con el objetivo recortado de PPO
 - 6: **end for**
 - 7: Evaluar el checkpoint sin actualizar sus parámetros
 - 8: Guardar efectivo, posiciones, operaciones y recompensa media
-

7.2 Validación técnica y pruebas

Las pruebas no se tratan como material auxiliar, sino como parte de la evidencia experimental del proyecto. Antes de interpretar resultados de modelos o agentes, se comprueba que las piezas que producen esos resultados funcionan de forma aislada: reglas económicas, parsing de mercado, persistencia, construcción de datasets, inferencia supervisada, simulador de cartera, restricciones de riesgo, entorno PettingZoo, adaptador RLlib, comandos de episodios y panel web.

La suite automatizada se organiza principalmente con Pytest y se complementa con Ruff y Mypy. Las pruebas unitarias cubren el comportamiento determinista del simulador, las reglas económicas del procedimiento manual, el OCR, SteamDT, Steam, BUFF, los datasets supervisados, el dataset de trading, el servicio de scoring y los componentes MARL. También existe una prueba de integración para repositorios de persistencia, separada porque depende de una base de datos disponible.

Las pruebas de rendimiento se integran en la evaluación del capítulo porque condicionan la utilidad del sistema. En este proyecto el rendimiento no se mide solo por tiempo de ejecución, sino también por cobertura de datos, número de señales generadas, coste de entrenamiento, capital bloqueado, drawdown y operaciones rechazadas. Los smoke tests de entrenamiento y los episodios MARL registran estas métricas para comprobar que el sistema no solo ejecuta, sino que produce salidas interpretables y comparables entre configuraciones.

Bloque	Qué verifica
Reglas y cartera	Fees, conversión, <i>trade hold</i> , posiciones y límites.
Datos y scraping	Parsing, normalización y construcción de históricos.
Modelo supervisado	Variables, carga de modelo e inferencia.
Simulación MARL	Acciones, observaciones, recompensas y adaptación a RLib.
Interfaz y operación	Estado, recomendaciones y ejecución de tareas.

Tabla 7.2: Cobertura de pruebas.

7.3 Métricas

Las métricas permiten interpretar una estrategia más allá de si produce una señal. Se separan indicadores de retorno y de riesgo para comprobar tanto el posible margen como el uso de capital y las restricciones que condicionan una decisión.

7.3.1 Métricas de rentabilidad

Las métricas de rentabilidad principales son beneficio neto, retorno porcentual, saldo final y número de señales operativas. En los modelos supervisados se usan además precisión, recall, F1, ROC-AUC, PR-AUC y Brier score. Para el uso operativo se da especial importancia a la precisión en umbrales altos, porque una señal menos frecuente pero más fiable puede ser más útil que muchas señales ruidosas. La calibración de probabilidades es relevante porque la salida del modelo se usa como señal de decisión, no solo como etiqueta binaria [21].

7.3.2 Métricas de riesgo

Las métricas de riesgo incluyen drawdown, capital bloqueado medio, posiciones abiertas, exposición por activo o plataforma, operaciones rechazadas por riesgo e impacto del *trade hold*. Estas métricas son necesarias para evaluar si una estrategia es operativamente utilizable y no solo rentable en una métrica aislada.

7.4 Resultados

Esta sección reúne los resultados obtenidos durante la construcción del sistema. Se presentan primero las comprobaciones sobre reglas y modelos y después las pruebas con datos de trading, OCR y agentes cooperativos.

7.4.1 Migración de reglas económicas

La primera fase ha consistido en convertir reglas del procedimiento manual en funciones comprobables. Se han migrado conversiones EUR/CNY, factores de comisión, cálculo de beneficio neto, rentabilidad porcentual, fecha de desbloqueo y capital bloqueado. Esta fase no produce un resultado de rentabilidad por sí misma, pero es necesaria para que los experimentos posteriores midan el mismo dominio que se utilizaba manualmente.

El resultado principal de esta fase es metodológico: las reglas económicas pasan de estar dispersas en fórmulas de hoja de cálculo a estar encapsuladas en módulos de simulación, con pruebas unitarias y parámetros explícitos. Esto reduce errores silenciosos y permite repetir un experimento con la misma configuración.

7.4.2 Resultados del modelo supervisado

La fase supervisada conserva una exploración de dirección Steam con 47.467 filas de entrenamiento, 19.252 de validación y 18.412 de prueba. Su etiqueta indica dirección de precio, no beneficio neto entre plataformas. Se mantiene como evidencia de que el pipeline de variables, calibración y selección funciona con una muestra de mayor tamaño, pero no se utiliza para aprobar compras. El protocolo principal de este capítulo es el corte BUFF–Steam que empieza en diciembre de 2025.

Umbral	Precisión	Recall	Señales
0,50	0,716	0,116	1.293
0,70	0,856	0,019	180
0,80	0,947	0,011	95
0,85	0,962	0,006	53
0,90	1,000	0,003	22

Tabla 7.3: Umbrales de dirección.

El umbral 0,85 deja 53 señales de las 18.412 filas de prueba. Esto ilustra el intercambio entre precisión y cobertura: elevar el umbral selecciona menos casos y no convierte la probabilidad en una orden. La calibración se revisa porque una probabilidad solo es útil si representa de forma razonable la frecuencia observada [21, 22]. Estos resultados no se consideran definitivos. Sí indican que el pipeline de entrenamiento, calibración, selección por umbral y evaluación temporal funciona.

7.4.3 Evolución de los datasets de trading

El dataset de compraventa no apareció en una única ejecución. Se han construido varios cortes porque cada uno reveló una limitación diferente del histórico disponible. El primer `trading_profit_v1` reunió 2.331 filas y 50 artículos. Sirvió para comprobar el constructor de etiquetas, las comisiones y la persistencia de características, pero la dirección analizada solo conservaba un caso positivo en validación y otro en prueba. Por ese motivo sus métricas no se comunican como resultado de un modelo.

Después se corrigieron el parser de BUFF, la normalización de moneda y el backfill de puntos históricos. Con estas correcciones se ejecutaron tres cortes BUFF–Steam desde diciembre de 2025. La Tabla 7.4 deja visibles tanto las muestras que permiten una exploración como las que se descartan. Esta distinción es importante: un dataset grande no es suficiente si sus bloques de validación solo contienen una clase.

La secuencia mejora la cobertura temporal, pero no el equilibrio de etiquetas. El conjunto inicial contiene 50 artículos, pero solo conserva un caso positivo en validación y otro en prueba. En marzo, la proporción de casos positivos es 93,0 % en entrenamiento, 98,7 % en validación y 89,5 % en prueba. Es el único corte que contiene ambas clases en los tres bloques, aunque sigue muy concentrado en oportunidades positivas.

Corte	Entrenamiento	Validación	Prueba
Inicial	2.331	—	—
Marzo	171	76	95
Mayo	361	76	95
Reciente	551	76	84

Tabla 7.4: Cortes de datos.

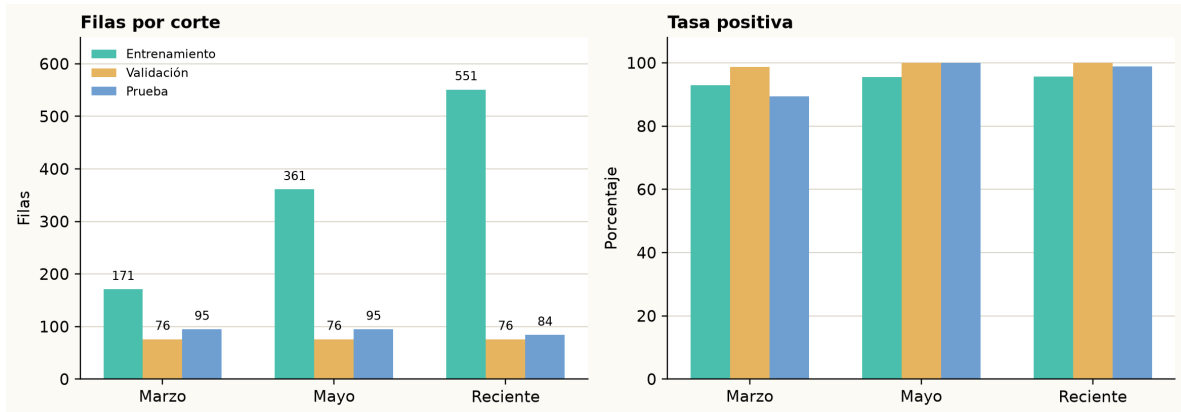


Figura 7.1: Evolución de cortes.

Los cortes de mayo y reciente amplían el número de filas. En mayo, validación y prueba contienen únicamente casos positivos. En el corte reciente las proporciones son 95,6 %, 100 % y 98,8 %, respectivamente. Estas muestras no habilitan ROC-AUC ni calibración sobre los bloques finales, por lo que se conservan como diagnósticos de calidad de datos y no como experimentos de rendimiento.

El flujo reproducible queda definido así: capturar observaciones, normalizar artículo y moneda, construir la etiqueta a ocho días, eliminar la zona de purga, separar por fecha y revisar la distribución antes de entrenar. Solo cuando cada bloque incluye resultados favorables y desfavorables se entrena un clasificador. Esta comprobación previa evitó presentar un entrenamiento trivial como una mejora del sistema.

7.4.4 Protocolo temporal reproducible

La validación se ha repetido el 10 de agosto de 2026 con los datos disponibles desde el 1 de diciembre de 2025. Cada ejemplo representa la posibilidad de comprar un artículo en BUFF y venderlo en Steam tras un horizonte de ocho días. La etiqueta vale uno cuando, después de aplicar las comisiones configuradas, el retorno supera el 10 %. El precio de compra se toma de BUFF y el precio de venta de Steam se valora con el factor de venta de 0,87 empleado por el simulador.

El orden temporal se conserva en todas las fases. Para el corte de marzo se usaron 171 ejemplos de entrenamiento, 76 de validación y 95 de prueba. El entrenamiento termina el 20 de enero, la validación ocupa del 2 al 20 de febrero y la prueba comprende del 4 al 28 de marzo. Se eliminan ocho días antes de cada frontera para que el horizonte futuro de una observación no alcance el bloque siguiente. Ningún parámetro se modifica

al consultar la prueba. Esta separación evita mezclar pasado y futuro, una precaución esencial cuando las observaciones tienen dependencia temporal [23].

La Figura 7.2 hace visible una limitación que no se aprecia al mirar solo el número de filas. Los tres bloques contienen las mismas 19 representaciones de artículo y están dominados por la clase rentable. La separación temporal evita que se use información futura, pero no demuestra por sí sola que el modelo generalice a artículos que no ha visto durante el entrenamiento.

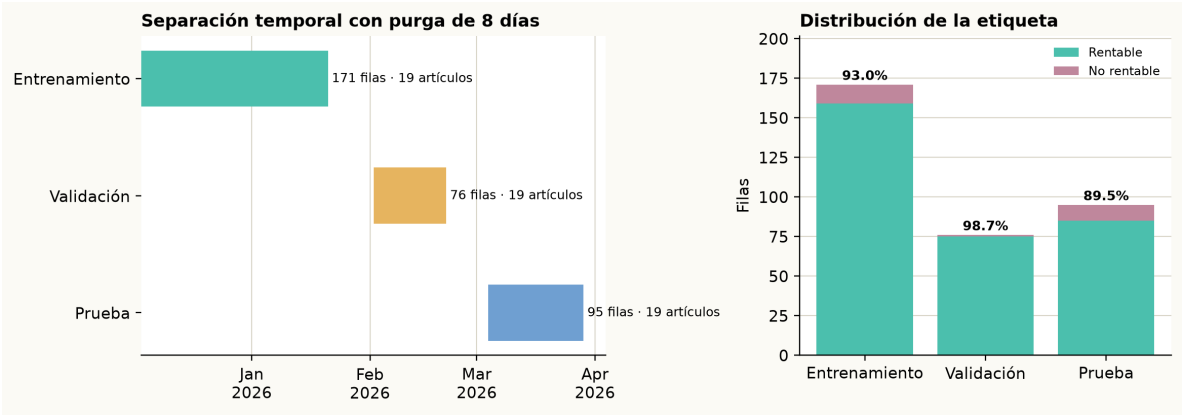


Figura 7.2: Corte temporal de marzo.

La selección compara las familias dummy, regresión logística, random forest e histogram gradient boosting. La configuración se elige en validación mediante precisión al umbral 0,85 con al menos tres señales. Después se calibra con sigmoide y se mide sobre el bloque de prueba. Además, se ejecuta una ablación de la regresión logística sin retardos, cambios, retornos ni medias móviles de uno, tres y siete días. Esta comparación responde a una pregunta concreta: si el histórico añade información o solamente complejidad. La precisión se presenta junto a la cobertura porque una clase mayoritaria puede hacer que el ROC-AUC parezca más concluyente de lo que permite la muestra [24].

La exploración no se limitó a escoger una familia. La Tabla 7.5 recoge el espacio de búsqueda empleado. El conjunto de prueba se dejó fuera de esta selección: sus valores se consultaron una vez después de fijar la configuración de validación.

Familia	Configuraciones exploradas
Regresión logística	Regularización L2 con $C \in \{0,3,1,3\}$.
Random forest	Profundidad 10 o 18, 160 árboles y cinco muestras por hoja.
HGB	Tasa de aprendizaje 0,05 o 0,10, 160 iteraciones y 31 hojas.

Tabla 7.5: Configuraciones de marzo.

La exploración inicial señala que la regresión logística sin características históricas puede separar mejor las dos clases que las alternativas probadas. También muestra que los retardos y medias móviles no deben darse por útiles sin contrastarlos. La siguiente ablación vuelve a medir esa decisión junto con la aumentación, usando particiones por fecha para que una fecha completa no se divida entre lados de un pliegue.

Configuración	ROC-AUC	Brier	Precisión @0,85	Señales
Logística con histórico	0,641	0,075	0,929	85
Logística sin histórico	0,760	0,072	0,970	67
Random forest con histórico	0,742	0,074	0,942	86
HGB con histórico	0,649	0,082	0,915	82

Tabla 7.6: Comparativa de marzo.

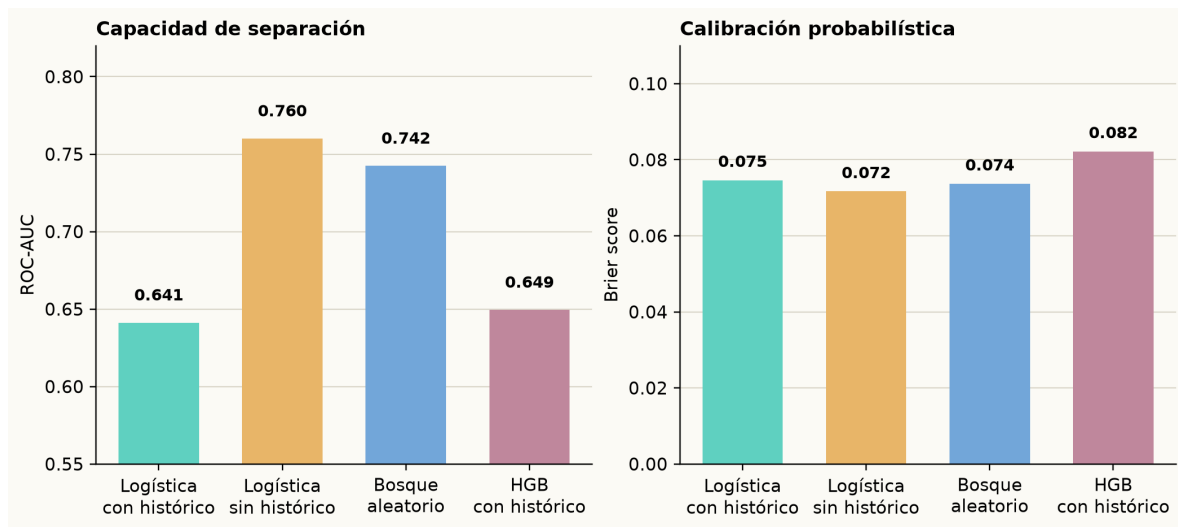


Figura 7.3: Modelos en marzo.

7.4.5 Ablación de histórico y aumentación

La segunda exploración usa una matriz 2×2 con regresión logística. Se mantienen los mismos cortes temporales de marzo, el umbral de selección 0,85, tres pliegues temporales y calibración sigmoide. Se comparan dos decisiones de diseño: conservar o retirar retardos y medias móviles, y entrenar con o sin aumentación numérica. La aumentación se contrasta como hipótesis y no se presupone beneficiosa, ya que su efecto depende del dominio y del volumen de datos [25].

La aumentación se aplica únicamente al entrenamiento. Para cada variable numérica se genera una copia perturbada mediante

$$x'_k = x_k + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, 0,01 \cdot IQR(x_k)).$$

Las etiquetas, las variables categóricas y las fechas no cambian. Además, todas las filas de una misma fecha se mantienen dentro del mismo lado de cada pliegue temporal. De este modo la copia perturbada no puede aparecer en entrenamiento mientras su original está en validación. La validación y la prueba permanecen sin modificar.

La combinación sin histórico ni aumentación obtiene el mejor equilibrio entre separación y calibración. El jitter duplica el tamaño de entrenamiento de 171 a 342 filas, pero empeora ROC-AUC y Brier en ambas configuraciones. Por ello no se incorpora al modelo de referencia. Este resultado no prueba que la aumentación sea perjudicial en cualquier muestra. Indica que, con una colección corta y muy concentrada en

Histórico	Aumentación	ROC-AUC	Brier	Precisión @0,85	Señales
Sí	No	0,631	0,076	0,931	87
No	No	0,716	0,076	0,952	62
Sí	Jitter	0,575	0,085	0,924	92
No	Jitter	0,613	0,084	0,919	86

Tabla 7.7: Ablación logística.

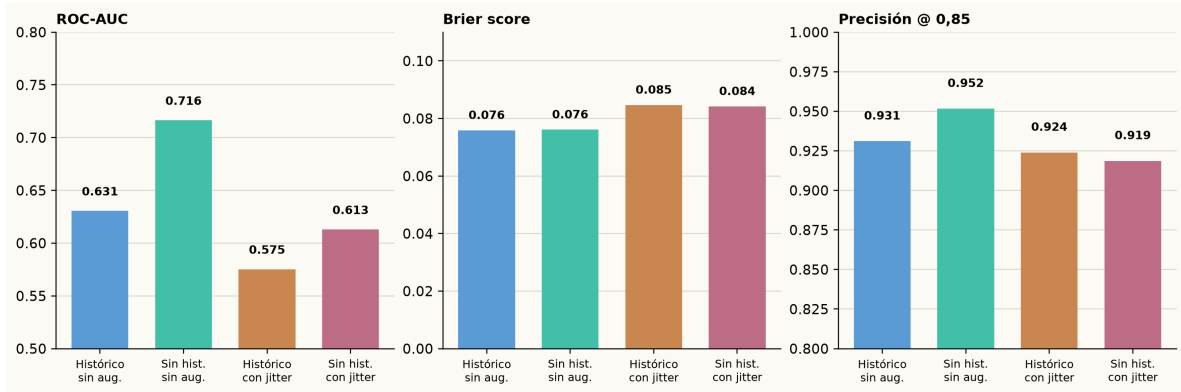


Figura 7.4: Histórico y aumentación.

operaciones positivas, introducir pequeñas variaciones de precio añade más ruido que información.

La lectura debe mantenerse prudente. Los 19 artículos disponibles en el corte aparecen tanto en entrenamiento como en prueba y la etiqueta positiva representa el 89,5 % de la prueba. Una estrategia que marca muchos casos como favorables puede alcanzar precisión elevada por la propia distribución de la muestra. El corte de mayo confirma este problema: de 361 ejemplos de entrenamiento, 76 de validación y 95 de prueba, las dos últimas particiones contienen exclusivamente etiquetas positivas. En esas condiciones ROC-AUC no es una métrica definida y no se entrena un modelo para presentarlo como comparación válida. Este resultado establece una condición de continuación clara: ampliar la cobertura cruzada BUFF–Steam y equilibrar los eventos antes de convertir el modelo en recomendación operativa.

La Figura 7.5 complementa esta lectura. La precisión se mantiene cerca de la tasa positiva de la prueba mientras el umbral aumenta, pero el número de señales solo cae de forma apreciable a partir de 0,90. Por tanto, un umbral alto no convierte por sí mismo la señal en evidencia de selección robusta. La calibración y la precisión deben volver a evaluarse cuando haya una proporción relevante de operaciones no rentables.

7.4.6 Comprobación del corte reciente

Con la base de datos conectada se repitió la construcción del dataset de BUFF a Steam el 11 de agosto de 2026. Se empleó la ventana desde diciembre de 2025, un horizonte de ocho días, una purga de ocho días y una prueba acotada a julio de 2026. El conjunto resultante conserva los 19 artículos ya presentes en el corte de marzo.

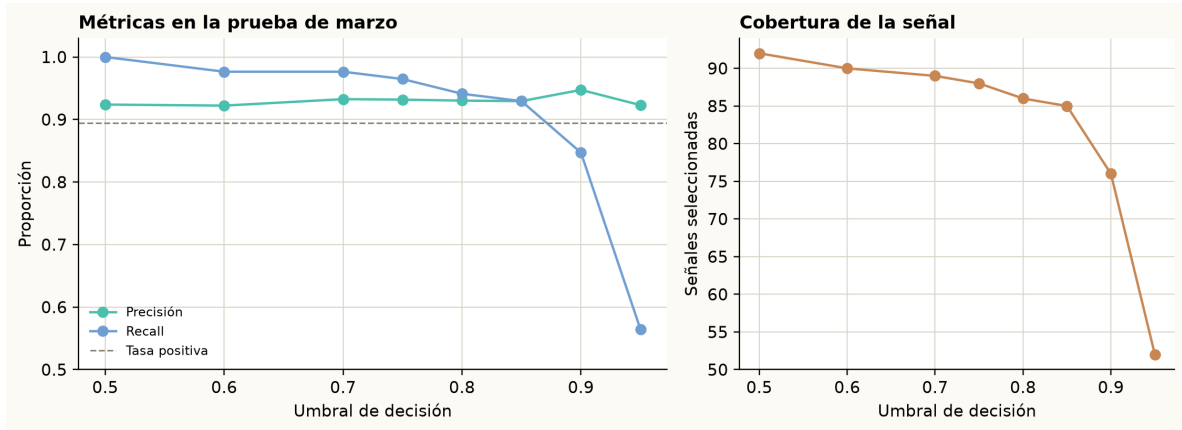


Figura 7.5: Métricas por umbral.

El conjunto no permite entrenar otro clasificador y sirve para comprobar si la nueva captura ha aportado variedad suficiente de etiquetas.

Entrenamiento	Validación	Prueba
551 (95,6 %)	76 (100 %)	84 (98,8 %)

Tabla 7.8: Corte BUFF–Steam.

El resultado es deliberadamente incómodo, pero es importante. En validación y prueba falta una clase negativa, por lo que ROC-AUC, calibración y una comparación de políticas no son interpretables. Entrenar con esta muestra produciría una clasificación trivial y no aportaría evidencia sobre el sistema. La prioridad experimental pasa a ser mantener capturas recurrentes de más artículos, incluir oportunidades descartadas y conservar decisiones cerradas para que la recompensa MARL pueda distinguir compras acertadas, compras fallidas y decisiones de espera.

7.4.7 Evidencia de pruebas automatizadas

La suite completa se ejecutó el 14 de agosto de 2026 y obtuvo 282 pruebas superadas en 24,94 segundos. Las tres pruebas de integración utilizaron el DATABASE_URL PostgreSQL configurado en Supabase. Registran un snapshot de mercado, comprueban la actualización de una cotización y guardan una señal de oportunidad vinculada a un artículo. Cada una abre una transacción externa y la revierte al terminar, por lo que la comprobación no añade datos de prueba al histórico utilizado por los experimentos. Este registro de datos, configuración y resultados permite repetir las comprobaciones, práctica recomendada para hacer evaluables los resultados de aprendizaje automático [26].

7.4.8 Validación funcional de OCR

La evaluación de OCR disponible es funcional, no una medida de exactitud sobre un corpus etiquetado de capturas. Por ello no se comunica una tasa de reconocimiento que no se ha medido. Las pruebas comprueban que el parser convierte una observación de mercado en un contrato con artículo, calidad, plataforma, precio, moneda y volumen,

Bloque	Pruebas	Alcance
Unitarias actuales	279	Economía, OCR, conectores, scraping, datasets, simulador, MARL y panel web.
Integración	3	Persistencia de snapshots, actualización de precios y señales en PostgreSQL.
Suite completa	282	Todas superadas en 24,94 segundos.

Tabla 7.9: Pruebas automatizadas.

que rechaza una confianza inferior a 0,5 y que el recorrido de imagen llama a carga, preprocesado y reconocimiento antes de persistir el resultado. El uso del motor se apoya en la arquitectura de Tesseract descrita por Smith [27]. La Tabla 7.10 delimita qué evidencia existe y qué queda pendiente.

Prueba	Caso ejecutado
Fixture de texto	Texto Steam y BUFF convertido a observaciones.
Umbral de confianza	Confianza 0,2 con mínimo configurado en 0,5.
Recorrido de imagen	Carga, preprocesado y reconocimiento con un runner controlado.

Tabla 7.10: Validación funcional de OCR.

La fixture confirma que el parser construye observaciones con los campos esperados a partir de texto de mercado. La prueba de confianza rechaza el valor 0,2 al exigir un mínimo de 0,5. El recorrido de imagen completa la secuencia de carga, preprocesado y reconocimiento y devuelve una confianza de 0,91 en el caso preparado.

Estas comprobaciones validan la integración, no la exactitud del OCR sobre capturas de interfaz. Por eso el siguiente experimento debe usar un conjunto etiquetado y comunicar por separado la exactitud de artículo, calidad, plataforma, precio y moneda. Hasta disponer de ese conjunto, el OCR se presenta como un conector validado y no como un sistema de reconocimiento cuantificado.

7.4.9 Dataset de trading operativo

El dataset `trading_profit_v1` se ha generado desde `market_history_points` usando precios Steam/BUFF, comisiones y horizonte temporal. La primera versión ha producido 2.331 ejemplos y 50 artículos, pero con desbalance extremo en validación y test: solo un caso positivo en cada split para la dirección analizada. Por este motivo, las métricas de ese entrenamiento no se usan como evidencia de rendimiento operativo.

El análisis posterior ha mostrado que el histórico de BUFF ha empezado a estar mejor alineado tras corregir el parser y permitir backfill de puntos históricos. Aun así, la dirección más natural de arbitraje, comprar en BUFF y vender en Steam, todavía no genera ejemplos suficientes a ocho días porque falta histórico futuro Steam alineado con los puntos BUFF recientes.

A partir de esta limitación, la validación distingue entre aprendizaje temporal e inferencia operativa: Steam se usa como fuente principal para aprender el riesgo temporal

de salida y BUFF queda como precio vivo de entrada en inferencia. El objetivo no es predecir el precio exacto de BUFF. La recomendación evalúa si un precio BUFF puntual conserva margen suficiente frente a una salida Steam estimada y sus comisiones.

7.4.10 Dificultades y pivotaciones

Durante el desarrollo se han producido varias decisiones de pivotaje. La primera aparece al formalizar el procedimiento manual: el valor principal estaba en extraer reglas económicas verificables y convertirlas en simulación reproducible. Esta decisión desplaza el trabajo desde una herramienta manual hacia un sistema auditable.

La segunda pivotación afecta a las fuentes de datos. El planteamiento inicial intentaba alinear históricos de Steam y BUFF para entrenar directamente sobre oportunidades de arbitraje entre plataformas. Las primeras pruebas mostraron dos problemas: falta de histórico alineado suficiente y riesgo operativo al consultar BUFF de forma reiterada. Por ello, el enfoque se ajusta: Steam se mantiene como fuente principal de aprendizaje temporal y BUFF se usa de forma puntual como precio vivo de entrada para evaluar flips concretos.

La tercera pivotación aparece en la parte multiagente. Antes de ampliar entrenamiento y episodios, se prioriza que el entorno, las recompensas, las restricciones y las métricas sean comprobables. Esta secuencia permite construir el entrenamiento sobre reglas económicas trazables, datos consistentes y comparaciones con baselines.

7.4.11 Protocolo de episodios MARL

La evaluación multiagente no utiliza una lista abstracta de acciones. Cada episodio recorre las observaciones del bloque de prueba de marzo en orden temporal. Scout indica si una oportunidad merece atención, Trader puede comprar una unidad, mantener o solicitar una venta y Portfolio aplica las restricciones de cartera. Una compra requiere el acuerdo de los tres roles. Una venta requiere que exista una posición desbloqueada del mismo artículo y que Portfolio la apruebe. De este modo, el episodio permite comprobar el intercambio entre una oportunidad local, las posiciones abiertas y el capital disponible para el conjunto de la cartera.

Elemento	Configuración comprobada
Observaciones	95 filas del bloque de prueba de marzo, en orden temporal.
Roles	Scout, Trader y Portfolio con observaciones locales.
Capital inicial	1.000 EUR en el recorrido de cartera de marzo.
Acciones	Compra con acuerdo de los tres roles y venta aprobada de posiciones desbloqueadas.
Estado de cartera	Efectivo, posiciones, exposición, bloqueo y drawdown.
Recompensa	Beneficio realizado, retorno, inactividad, restricciones, drawdown, bloqueo y volatilidad.
Smoke PPO	Una iteración y 64 pasos mediante el adaptador PettingZoo-RLlib.

Tabla 7.11: Episodio MARL.

La diferencia entre el recorrido funcional y PPO debe mantenerse clara. La regla determinista se usa para verificar el ciclo de cartera completo. El smoke de PPO comprueba que el mismo entorno puede entregar observaciones y recompensas a RLlib. Ninguno de estos dos pasos constituye todavía una comparación de políticas entrenadas.

7.4.12 Resultados MARL

El entorno multiagente se ha comprobado con el bloque de marzo. Scout marca una oportunidad, Trader solicita una compra o una venta y Portfolio decide si la operación respeta saldo, exposición, volumen de intercambios y capital bloqueado. La recompensa compartida utilizada en estas ejecuciones se define en la Sección 4.6.2. La venta añade un resultado importante: el beneficio se registra al cerrar la posición, no solo como una estimación en el momento de comprar.

Ejecución	Finalidad	Pasos
Caso controlado	Compra, bloqueo de ocho días y venta de una posición.	2
Recorrido de cartera	Regla de margen positivo con compra, espera y venta.	95
PPO, una iteración	PPO multiagente mediante PettingZoo-RLlib.	64

Tabla 7.12: Ejecuciones MARL.

En el caso controlado se compró una unidad por 10,00 EUR en BUFF el 1 de enero de 2026. La venta permaneció inhabilitada hasta el 9 de enero. Al cerrarla en Steam por 14,00 EUR brutos, el simulador aplicó la comisión de venta y registró un ingreso neto de 12,18 EUR. La cartera pasó de 100,00 EUR a 102,18 EUR y el beneficio realizado fue 2,18 EUR. Esta prueba confirma el cumplimiento del bloqueo y el registro de una venta, pero no evalúa una estrategia de selección.

El recorrido de cartera parte de 1.000 EUR y procesa 95 observaciones de marzo. La regla determinista compra cuando el margen neto observado es positivo y las restricciones lo permiten. Cuando encuentra una posición desbloqueada del mismo artículo, solicita su venta. Se ejecutaron 24 compras y 19 ventas, con un máximo de 11 posiciones simultáneas. Al terminar quedaron cinco posiciones abiertas, 306,60 EUR de capital bloqueado, 888,99 EUR de efectivo disponible y 195,59 EUR de beneficio realizado. El valor de cartera registrado fue 1.195,59 EUR.

Estas cifras verifican que el entorno conserva las posiciones abiertas, aplica límites y reutiliza efectivo después de una venta. No miden el rendimiento de PPO ni permiten estimar beneficio futuro: el corte contiene oportunidades preseleccionadas y está muy concentrado en márgenes positivos.

El comando PPO con RLlib completó una iteración y 64 pasos de entorno con el conjunto de entrenamiento. El estado central se expone al adaptador y hay una política por rol. La evaluación del checkpoint corto no ejecutó compras. Esta prueba valida la compatibilidad técnica, pero una única iteración no permite comparar una política

aprendida con una regla de margen. Las comparaciones futuras deberán repetir semillas, episodios y cortes temporales para comunicar la variabilidad del aprendizaje [28].

7.5 Experimentos finales pendientes

Para cerrar la validación del TFM será necesario ejecutar una fase experimental más estricta. Primero debe validarse el modelo Steam-only de salida a ocho días y conectarlo a candidatos con precio BUFF vivo. Después deben compararse baselines simples: comprar todos los candidatos de SteamDT, comprar solo cuando el margen BUFF $\rightarrow$ Steam es positivo, agente único y política dummy.

La evaluación MARL deberá incluir ablations. Las más relevantes son: sin probabilidad supervisada, sin agente Portfolio, sin penalización por capital bloqueado y sin restricciones de volumen de intercambios. Estas comparaciones permitirán distinguir si la mejora viene de la arquitectura multiagente, del modelo supervisado, de una regla de riesgo o simplemente de la estructura del simulador.

7.6 Discusión

La discusión relaciona los resultados con el alcance del trabajo. Se identifican las condiciones que limitan su interpretación, las partes que ya funcionan de forma integrada y los datos que faltan para una evaluación más sólida.

7.6.1 Amenazas a la validez

La principal amenaza interna es el desbalance de etiquetas: la prueba de marzo contiene 85 casos rentables de 95 y los bloques posteriores llegan a tener una sola clase. La segunda es el solapamiento de artículos entre entrenamiento y prueba. La división por fecha protege frente al uso de futuro, pero no responde a la pregunta más exigente de generalizar a activos virtuales no presentes en el entrenamiento. La tercera amenaza es operativa: el precio BUFF se usa como entrada puntual y la salida Steam se estima con histórico y comisiones, por lo que la sincronización exacta de ambos mercados requiere más capturas. Además, los experimentos simulan operaciones con datos observados. No envían órdenes reales, no transfieren artículos entre plataformas ni utilizan capital real, por lo que no permiten garantizar la ejecución futura de una operación.

También hay límites de medición. El OCR está validado a nivel funcional, pero aún no dispone de un corpus de imágenes etiquetado. El entorno MARL ya incluye compra, bloqueo y venta simulada, aunque todavía no cuenta con una política entrenada durante suficientes episodios. Estas limitaciones no invalidan la arquitectura, pero acotan las conclusiones a viabilidad técnica y experimental preliminar.

7.6.2 Casos favorables

El trabajo realizado muestra que el sistema ya puede integrar fuentes, construir datasets, entrenar modelos supervisados, calibrar probabilidades, simular reglas económicas y ejecutar un smoke multiagente. La separación entre adquisición, predicción, simulación y decisión ha resultado útil porque permite avanzar por partes y detectar bloqueos concretos sin rehacer toda la arquitectura.

7.6.3 Limitaciones observadas

La principal limitación actual es la disponibilidad de histórico alineado entre Steam y BUFF. Sin suficientes ejemplos negativos en validación y prueba, cualquier métrica de trading puede ser engañosa. También existe riesgo de sobreajuste en variables categóricas de alta cardinalidad y en señales de precisión alta con pocos casos. Las reglas de las plataformas, sus comisiones y la actividad de intercambio también pueden cambiar con el tiempo, por lo que los resultados deben interpretarse según las condiciones observadas en cada captura. Por último, el entorno MARL necesita una validación experimental más amplia antes de poder concluir si la arquitectura multiagente mejora a una estrategia simple.

7.6.4 Lectura de los resultados

Los resultados confirman la viabilidad técnica de la integración y la simulación. La comparación del rendimiento económico entre políticas multiagente requiere una fase experimental adicional con más episodios, datos y baselines homogéneos.

Conclusiones

Este capítulo resume las aportaciones del trabajo, revisa el grado de cumplimiento de los objetivos y delimita los pasos necesarios para ampliar la validación experimental y el entrenamiento multiagente.

8.1 Conclusiones principales

El desarrollo realizado hasta el momento permite concluir que el problema no puede reducirse a una predicción aislada de subida o bajada de precio. En mercados como el de *Counter-Strike 2*, la decisión depende de fricciones muy concretas: comisiones, conversión de moneda, volumen de intercambios, *trade hold*, capital bloqueado y riesgo de cartera. Por ello, la arquitectura propuesta necesita combinar adquisición de datos, simulación económica y decisión multiagente, siguiendo la disciplina de backtesting y control de fricciones propia de los entornos de trading con aprendizaje automático [17].

También se ha comprobado que la separación por capas facilita el avance del proyecto. Los agentes extractores se encargan de observar el mercado, el modelo supervisado produce señales probabilísticas, el simulador aplica reglas económicas y los agentes MARL operan dentro de un entorno controlado. Esta división reduce el acoplamiento y permite validar cada parte por separado.

La contribución principal del desarrollo realizado no es afirmar que el sistema gane dinero, sino construir una base reproducible para estudiar si el sistema puede seleccionar operaciones rentables bajo las restricciones del mercado. El proyecto ya dispone de una arquitectura modular, un flujo de adquisición, un modelo de persistencia, un simulador económico, datasets derivados, modelos supervisados preliminares y un entorno multiagente funcional para smoke tests.

Una conclusión metodológica relevante es que el proyecto ha evolucionado a partir de las dificultades encontradas. La falta de histórico Steam/BUFF suficientemente alineado, el riesgo de scraping reiterado y el desbalance de los primeros datasets de trading han llevado a separar entrenamiento e inferencia operativa. La separación entre entrenamiento e inferencia no cambia el objetivo del sistema, pero sí mejora su viabilidad: el aprendizaje temporal se apoya en Steam y la comparación con BUFF se reserva para evaluar oportunidades concretas.

8.2 Cumplimiento de objetivos

Se ha avanzado en la mayoría de objetivos técnicos: existe un monorepo modular, un modelo de datos operativo, pipelines de adquisición, análisis del procedimiento manual inicial, construcción de datasets, entrenamiento supervisado, simulador de cartera y un entorno MARL funcional para pruebas cortas. También se han definido agentes Scout, Trader y Portfolio, junto con restricciones de riesgo y métricas de evaluación.

El cumplimiento experimental todavía es parcial. Los resultados supervisados muestran que el pipeline funciona y que algunos modelos producen señales de alta precisión en umbrales exigentes, pero el número de señales es limitado. El dataset operativo de trading Steam/BUFF está construido, aunque sus cortes presentan desbalance extremo en validación y test. El entorno MARL ya ha completado ciclos de compra, bloqueo y venta con varias posiciones, pero no constituye todavía una evaluación comparativa completa. Las pruebas automatizadas y los smoke tests de rendimiento forman parte del capítulo experimental porque verifican que las métricas obtenidas proceden de componentes comprobados.

8.3 Limitaciones

La limitación más importante es la escasez de histórico alineado entre plataformas para algunas direcciones de trading. El dataset más cercano al objetivo operativo carece de suficientes ejemplos negativos en validación y test, por lo que no permite defender todavía una mejora robusta. Además, algunos resultados supervisados tienen alta precisión en umbrales altos, pero con pocas señales, lo que obliga a interpretarlos con prudencia. Por ello, se separa el aprendizaje temporal basado en Steam del uso puntual de BUFF como precio vivo de entrada para flips.

Otra limitación es que el smoke multiagente demuestra que el flujo ejecuta, pero no que el sistema haya aprendido una política superior. Para llegar a esa conclusión se necesita una evaluación reproducible con más episodios, más datos, baselines justos y métricas de riesgo.

También existe una limitación propia del dominio. Los activos de CS2 dependen de plataformas externas, reglas de intercambio, sesiones, cambios visuales y restricciones que pueden variar. Por tanto, cualquier sistema automatizado debe mantener trazabilidad, parámetros versionados y capacidad de revisar errores de fuente.

8.4 Trabajo futuro

Las líneas inmediatas de trabajo son validar un modelo Steam-only de salida a ocho días, usar BUFF como precio vivo de entrada en inferencia de flip, comparar contra baselines simples y entrenar MARL con una configuración CTDE completa. Después será necesario estudiar ablations, generar gráficas de resultados y cerrar una discusión experimental sobre cuándo la arquitectura multiagente aporta valor y cuándo no.

Los experimentos finales deberán reportar beneficio neto, saldo final, drawdown, capital bloqueado, número de operaciones, posiciones abiertas y rechazos por riesgo. Además, deberán comparar la arquitectura completa contra versiones simplificadas: sin probabilidad supervisada, sin agente Portfolio y con reglas simples de spread. Solo con

esa comparación será posible defender si la descentralización multiagente aporta una mejora frente a estrategias más sencillas, siguiendo la lógica comparativa habitual en MARL y entrenamiento cooperativo [14, 15].

Bibliografía

- [1] Vili Lehdonvirta. Virtual item sales as a revenue model: Identifying attributes that drive purchase decisions. *Electronic Commerce Research*, 9(1):97–113, 2009.
- [2] Yue Guo and Stuart J. Barnes. Virtual item purchase behavior in virtual worlds: An exploratory investigation. *Electronic Commerce Research*, 9(1–2):77–96, 2009.
- [3] Valve. Steam community market faq. <https://help.steampowered.com/en/faqs/view/61F0-72B7-9A18-C70B>. Consultado en agosto de 2026.
- [4] Roblox. The marketplace. <https://en.help.roblox.com/hc/en-us/articles/203313300-The-Marketplace>. Consultado en agosto de 2026.
- [5] Blizzard Entertainment. Using a wow token. <https://us.support.blizzard.com/en/article/31218>. Consultado en agosto de 2026.
- [6] CCP Games. Buy and sell orders. <https://support.eveonline.com/hc/en-us/articles/203218932-Buy-and-Sell-Orders>. Consultado en agosto de 2026.
- [7] BUFF. Buff market. <https://buff.163.com/market/csgo>. Consultado en agosto de 2026.
- [8] Skinport. Items | skinport rest api. <https://docs.skinport.com/items>. Consultado en agosto de 2026.
- [9] BitSkins. Bitskins market. <https://bitskins.com/market/skin/>. Consultado en agosto de 2026.
- [10] DMarket. Dmarket marketplace. <https://dmarket.com/marketplace>. Consultado en agosto de 2026.
- [11] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [12] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2 edition, 2009.
- [13] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- [14] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.
- [15] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative, multi-agent games. In *Advances in Neural Information Processing Systems*, 2022.

- [16] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [17] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the Second ACM International Conference on AI in Finance*, 2021.
- [18] SteamDT. Steamdt: Cs2 market overview. <https://www.steamdt.com/en>. Consultado en agosto de 2026.
- [19] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [20] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [21] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632, 2005.
- [22] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1321–1330. PMLR, 2017.
- [23] Christoph Bergmeir, Rob J. Hyndman, and Bonsoo Koo. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83, 2018.
- [24] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.
- [25] Brian Kenji Iwana and Seiichi Uchida. An empirical survey of data augmentation for time series classification with neural networks. *PLOS ONE*, 16(7):e0254841, 2021.
- [26] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021.
- [27] Ray Smith. An overview of the Tesseract OCR engine. In *Ninth International Conference on Document Analysis and Recognition*, volume 2, pages 629–633, 2007.
- [28] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320, 2021.
- [29] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. <https://arxiv.org/abs/1707.06347>, 2017.

- [30] Justin K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Rodrigo Perez, Caroline Horsch, Clemens Diefendahl, Niall L. Williams, Yashas Lokesh, Praveen Ravi Hines, and Niall Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2021.
- [31] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.

Anexos

Los anexos reúnen detalles operativos que complementan la explicación principal: estructura del repositorio, configuración del entorno, comandos reproducibles y parámetros de entrenamiento.

A.1 Estructura del repositorio

El código fuente completo se encuentra en el repositorio público del proyecto:

<https://github.com/STOSKR/TFM-CSFlipper>

La Figura A.1 resume la organización. Se muestran solo los directorios que ayudan a localizar cada parte del sistema, mientras que los ficheros y resultados generados se consultan directamente en el repositorio.

TFM-CSFlipper/	
apps/	comandos, extractores e interfaz local
packages/	lógica reutilizable del sistema
tests/	pruebas unitarias y de integración
data/	datasets y resultados derivados
docs/	documentación técnica
TFM/	memoria, figuras y bibliografía

Figura A.1: Estructura del repositorio.

Los directorios principales cumplen estas funciones:

- **apps**: contiene los extractores, los comandos de consola y la interfaz local.
- **packages**: agrupa los contratos, la persistencia, la predicción, el simulador, el entorno MARL y el procesamiento visual.
- **tests**: reúne las pruebas unitarias y de integración que verifican reglas económicas, conectores y flujos principales.
- **data**: conserva datasets derivados, informes y resultados que no forman parte del código fuente.
- **docs**: recoge documentación técnica y decisiones de desarrollo.
- **TFM**: contiene el código $\text{\LaTeX}$, las figuras y la bibliografía de esta memoria.

A.2 Configuración del entorno

El entorno se define en `pyproject.toml`. El proyecto usa Python, Supabase/Postgres, Pytest, Ruff, Mypy y dependencias opcionales para MARL como Ray/RLlib, PettingZoo, Gymnasium y PyTorch. Para desarrollo local existe un `docker-compose.yml` con Postgres.

Comandos representativos:

```
python -m pytest tests/unit
python -m ruff check
python -m mypy apps packages tests/unit
python -m apps.cli.auto_scrape_loop --once
python -m apps.cli.render_stream_scrape
python -m apps.cli.web_dashboard_server
```

A.3 Comandos de datasets y modelos

Los comandos de datos y modelos separan adquisición, construcción de dataset, entrenamiento e inferencia. Esta separación permite repetir una fase sin rehacer todo el flujo.

```
python -m apps.cli.build_supervised_dataset
python -m apps.cli.analyze_supervised_coverage
python -m apps.cli.train_supervised_model
python -m apps.cli.build_trading_dataset
python -m apps.cli.retrain_trading_model
python -m apps.cli.score_live_opportunities
```

A.4 Parámetros de entrenamiento

Los experimentos supervisados usan splits temporales, calibración de probabilidades y selección por precisión operativa en umbrales altos. En los smoke tests se han comparado modelos dummy, regresión logística, random forest e histogram gradient boosting. El entrenamiento MARL usa RLLib con un entorno multiagente y tres políticas: Scout, Trader y Portfolio.

A.5 Modelo de datos

El modelo operativo actual gira alrededor de dos tablas principales: `market_items` y `market_history_points`. La primera representa artículos identificados por sus atributos y su estado actual por plataforma. La segunda almacena series temporales en formato largo, con una fila por artículo, plataforma, fecha y métrica. La separación entre `market_items` y `market_history_points` evita mezclar métricas distintas y permite añadir nuevas señales sin alterar la forma general del histórico.

El esquema canónico previsto añade entidades como `assets`, `platforms`, `market_observations`, `predictions`, `agent_actions`, `investment_decisions` y

Bloque	Parámetros relevantes
Supervisado	Split temporal, umbral operativo, calibración, modelo candidato y columnas excluidas por leakage.
Trading	Dirección de arbitraje, horizonte, fees, tipo de cambio, ventana histórica y target de beneficio neto.
Simulación	Saldo inicial, tamaño máximo de posición, <i>trade hold</i> , comisiones y límites de exposición.
MARL	Número de episodios, políticas Scout/Trader/Portfolio, algoritmo PPO y métricas de cartera.

Tabla A.1: Configuración experimental.

`simulated_positions`. El objetivo de este esquema es mejorar trazabilidad, auditoría y separación entre observación, predicción y decisión.

A.6 Resultados adicionales

Los resultados intermedios se guardan en `model-runs` y los datasets derivados en `data/datasets`. Estas carpetas permiten conservar reportes de cobertura, exploración de características, métricas de entrenamiento, umbrales seleccionados y salidas de smoke tests sin mezclar los artefactos pesados con el código principal.

Para que un experimento aporte evidencia reproducible, debe registrar como mínimo: configuración usada, periodo temporal, número de activos, tamaño de cada split, tasa de positivos, modelo o política evaluada, métricas principales y limitaciones observadas.